RTL Design Sherpa

RTL Clock Domain Crossing Module Reference — Rev 0.90

July 2026

Table of Contents

1 CDC Primer: Clock Domain Crossing Techniques

This document categorizes the clock domain crossing approaches available in RTL Design Sherpa and provides guidance on when to use each one. All modules listed here are production-ready with formal verification.

1.1 The Three Vector CDC Categories

When you need to move multi-bit data across clock domains, there are three fundamental approaches. They differ in how they guarantee that the destination samples stable data.

Category	Stability Guarantee	Throughput	Complexity	Use When
Open-Loop	Source holds data stable	Low (~1 transfer per 6 dst clocks)	Minimal	Config registers, infrequent updates
Closed-Loop Handshake	Explicit req/ack acknowledgment	Medium (~1 per 5-8 dst clocks)	Moderate	Control signals, command/response
Async FIFO	Gray-coded pointer synchronization	High (1 per dst clock sustained)	Higher	Streaming data, DMA, packet transfer

1.2 Category 1: Open-Loop CDC

Principle: The source asserts a single-cycle valid pulse and holds data stable. A synchronized copy of the valid pulse propagates to the destination domain, which latches the data when the pulse arrives. There is no acknowledge path – the source must guarantee the hold time.

When it works: The source naturally holds data for multiple destination clock periods. This is common for: - Fast source to slow destination (the fast domain's data persists for many slow clocks) - Configuration register writes (software writes once, hardware reads later) - Status flags that change infrequently

When it fails: If the source can change data before the destination has sampled it. Without an acknowledge, there is no backpressure.

1.2.1 Module: cdc_open_loop

```
// CPU at 100 MHz, peripheral at 25 MHz (ratio 4:1)
// STRETCH_CYCLES = ceil(1.5 * 4) + 3 = 9
cdc_open_loop #(
    .DATA_WIDTH     (32),
    .STRETCH_CYCLES (9),
    .SYNC_STAGES    (3)
) u_cfg_cdc (
    .clk_src   (cpu_clk),
    .rst_src_n (cpu_rst_n),
    .src_valid (cfg_write_pulse),   // Single-cycle pulse
    .src_data  (cfg_write_data),
    .src_busy  (cfg_busy),         // Blocks next write during stretch

    .clk_dst   (periph_clk),
    .rst_dst_n (periph_rst_n),
    .dst_valid (cfg_update_pulse),  // Single-cycle in dst domain
    .dst_data  (cfg_update_data)    // Latched, stable until next
update
);
```

Internals: Source captures data into a holding register and asserts a stretched valid level for `STRETCH_CYCLES` source clocks. The destination synchronizes the stretched level through `glitch_free_n_dff_arn` and latches data on the rising edge. `src_busy` blocks the next transfer during the stretch countdown.

Setting STRETCH_CYCLES: `STRETCH_CYCLES >= ceil(1.5 * T_dst / T_src) + SYNC_STAGES`. This ensures the stretched valid is seen by at least one destination clock edge after passing through the synchronizer. When in doubt, round up – extra cycles cost only source-side latency.

Documentation: cdc_open_loop.md

1.2.2 Supporting Primitives

Module	Purpose	Doc
glitch_free_n_dff_arn	N-stage synchronizer primitive with ASYNC_REG	rtl/common/
cdc_synchronizer	Multi-flop synchronizer	rtl/amba/shared/

Module	Purpose	Doc
	wrapper for quasi-static signals	
sync_pulse	Single-cycle pulse crossing (toggle + edge detect)	rtl/common/
reset_sync	Reset synchronizer (async assert, sync deassert)	rtl/common/

1.3 Category 2: Closed-Loop Handshake CDC

Principle: The source asserts a request and holds data stable. The destination synchronizes the request, samples the data, and sends back an acknowledge. The source waits for the acknowledge before releasing the data or starting a new transfer. This explicit round-trip guarantees data stability regardless of clock frequency ratios.

Two variants exist, differing in how the request/acknowledge signals encode "new transfer":

1.3.1 2-Phase (Toggle / NRZ)

Each new transfer is signaled by a **transition** (toggle) of the request bit. The destination detects the edge and toggles its acknowledge bit in response. Two synchronizer crossings per transfer (req toggle + ack toggle).

```
Source:  data stable, toggle req ----[sync]----> Destination sees
edge, latches data
                                                    toggle ack ----
[sync]----> Source sees edge, done
```

Advantage: Faster – half the control-path round trip of 4-phase. **Disadvantage:** Edge detection is slightly trickier to reason about.

1.3.2 4-Phase (Level / RZ)

Each transfer uses full rise-and-fall cycles: req rises, ack rises, req falls, ack falls. Four synchronizer crossings per transfer.

```
Source:  data stable, req=1 ----[sync]----> Destination sees req=1,
latches data
                                                ack=1 ----[sync]---->
Source sees ack=1, req=0
                                                ack=0 (after req=0
propagates) ----> Ready for next
```

Advantage: Simpler level-based logic, easier to verify. **Disadvantage:** Slower – four crossings instead of two.

1.3.3 Modules

Module	Variant	Latency	Doc
cdc_2_phase_handshake	Toggle (NRZ)	~5-6 dst clocks	rtl/amba/shared/
cdc_4_phase_handshake	Level (RZ)	~7-8 dst clocks	rtl/amba/shared/

Both modules have identical port interfaces and are drop-in interchangeable:

```
// Change only the module name to switch variants
cdc_2_phase_handshake #(   // or cdc_4_phase_handshake
    .DATA_WIDTH     (64),
    .SYNC_STAGES    (3),
    .TIMEOUT_CYCLES (1000)
) u_pkt_cdc (
    .clk_src     (mon_clk),
    .rst_src_n   (mon_rst_n),
    .src_valid   (pkt_valid),
    .src_ready   (pkt_ready),    // Backpressure to source
    .src_data    (pkt_data),
    .src_timeout (pkt_timeout),  // Stall detection

    .clk_dst     (sys_clk),
    .rst_dst_n   (sys_rst_n),
    .dst_valid   (rx_valid),
    .dst_ready   (rx_ready),
    .dst_data    (rx_data)
);
```

1.3.4 Protocol-Level CDC Using Handshakes

The handshake modules are building blocks for protocol-level CDC:

Module	Protocol	Handshake Used	Doc
apb_slave_cdc	APB4	Selectable 2-phase or 4-phase	rtl/amba/apb/
apb5_slave_cdc	APB5	Selectable 2-phase or 4-phase	rtl/amba/apb5/

1.4 Category 3: Gray-Coded Pointers / Async FIFO

Principle: Instead of synchronizing data directly, synchronize the read and write **pointers** of a dual-port memory. The pointers are Gray-coded so that only one bit changes per increment. This means any sample during a transition yields either the old or new pointer value – never a corrupted intermediate. The memory itself is dual-ported (one port per clock domain) and requires no synchronization.

This is the dominant technique for streaming data across clock domains. If you are moving continuous data rather than occasional transfers, this is what you reach for.

1.4.1 How It Works

```
Write Domain:                                    Read Domain:
  wr_ptr (binary) ---> bin2gray --->              gray2bin ---> Compare with
rd_ptr
                          [sync N stages]                       for empty
detection

  gray2bin <--- [sync N stages] <--- bin2gray <--- rd_ptr (binary)
  Compare with wr_ptr for full detection
```

1. Each domain maintains its own pointer in binary (for memory addressing)
2. Each pointer is converted to Gray code (for safe CDC crossing)
3. The Gray-coded pointer is synchronized into the other domain
4. Full/empty flags are derived by comparing the local pointer with the synchronized remote pointer
5. The dual-port memory is addressed by the binary pointers directly (no CDC needed for data)

1.4.2 Standard Gray Code vs Johnson Counter

The repository provides two async FIFO implementations that differ in how they encode pointers:

Standard Gray Code (`fifo_async`): Uses `bin2gray` / `gray2bin` converters with standard binary-reflected Gray code. Depth must be a power of 2 (because standard Gray codes only have single-bit transitions for power-of-2 counts).

Johnson Counter (`fifo_async_div2`): Uses a Johnson counter sequence, which is a subset of Gray code that supports even (non-power-of-2) depths. A Johnson counter of N bits cycles through 2N states, each differing by one bit. The `grayj2bin` converter decodes the Johnson sequence back to binary for pointer comparison.

FIFO	Pointer Encoding	Depth Constraint	Best For
fifo_async	Standard Gray code	Power of 2 only	Most use cases

FIFO	Pointer Encoding	Depth Constraint	Best For
fifo_async_div2	Johnson counter	Any even number	Odd-sized buffers, area optimization

1.4.3 Sizing the Depth

Given a write/read clock ratio and a burst pattern, the minimum FIFO depth to avoid dropping writes is:

```
depth >= burst_length * (1 - read_rate / write_rate)
```

where `rate = freq * duty_cycle` (words/second) for each side. Round the result up, then apply the encoding's depth constraint:

- **Gray (`fifo_async`):** round up to the next power of two.
- **Johnson (`fifo_async_div2`, `gaxi_fifo_async`):** round up to the next even number — the computed depth itself, in most cases.

Worked example. Writer at 100 MHz bursts 100 back-to-back writes; reader pulls 1 word/clk at 80 MHz:

Step	Value
Burst duration	100 / 100 MHz = 1 µs
Reads during burst	1 µs × 80 MHz = 80
Net build-up	100 − 80 = **20 words**
Gray FIFO (power-of-2)	round up to **32**
Johnson FIFO (even)	**20** as-is

The Gray encoding adds 12 unused slots — overhead that scales with the data width:

`DATA_WIDTH`	Gray depth 32	Johnson depth 20	Savings
32 b	1024 b	640 b	384 b (38%)
64 b	2048 b	1280 b	768 b (38%)
256 b	8192 b	5120 b	3072 b (38%)
1024 b	32 768 b	20 480 b	**12 288 b** (38%)

For wide streaming paths (≥256 b), picking Johnson when the rate analysis lands between two powers of two reclaims a meaningful slice of BRAM / register area. The downside is a slightly larger pointer comparator (Johnson pointers are `2 × log2(depth)` bits wide vs Gray's `log2(depth) + 1`), but for `DATA_WIDTH ≥ 64` the storage savings dominate.

Don't forget to add margin for synchronizer latency on `full`/`empty` if you're at the edge of the safe depth — the back-pressure to the writer lags real fill level by `N_FLOP_CROSS` write clocks.

1.4.4 Modules

Module	Description	Depth	Doc
fifo_async	Basic async FIFO, Gray-coded pointers	Power of 2	rtl/common/
fifo_async_div2	Async FIFO, Johnson counter pointers	Any even	rtl/common/
gaxi_fifo_async	GAXI-interface async FIFO	Configurable	rtl/amba/gaxi/
gaxi_skid_buffer_async	Pipelined async FIFO (skid + async)	Configurable	rtl/amba/gaxi/

1.4.5 Usage Example

```
fifo_async #(
    .DATA_WIDTH    (64),
    .DEPTH         (16),       // Must be power of 2
    .N_FLOP_CROSS  (3),        // Synchronizer stages
    .ALMOST_WR_MARGIN (2),     // Almost-full threshold
    .ALMOST_RD_MARGIN (2)      // Almost-empty threshold
) u_stream_cdc (
    // Write domain
    .wr_clk     (src_clk),
    .wr_rst_n   (src_rst_n),
    .wr_en      (src_valid && !wr_full),
    .wr_data    (src_data),
    .wr_full    (wr_full),
    .wr_almost_full (wr_almost_full),

    // Read domain
    .rd_clk     (dst_clk),
    .rd_rst_n   (dst_rst_n),
    .rd_en      (dst_ready && !rd_empty),
    .rd_data    (dst_data),
    .rd_empty   (rd_empty),
    .rd_almost_empty (rd_almost_empty)
);
```

1.4.6 Supporting Primitives

Module	Purpose	Doc
bin2gray	Binary to Gray code converter	rtl/common/

Module	Purpose	Doc
gray2bin	Gray code to binary converter	rtl/common/
grayj2bin	Johnson counter to binary converter	rtl/common/
counter_bingray	Dual binary+Gray output counter	rtl/common/
counter_johnson	Johnson counter for div2 FIFOs	rtl/common/
glitch_free_n_dff_arn	N-stage synchronizer for pointer crossing	rtl/common/

1.5 Decision Guide: Which CDC Technique?

```
Need to cross clock domains with multi-bit data?
|
+-- Is data streaming / continuous?
|   |
|   +-- YES --> Async FIFO (fifo_async or gaxi_fifo_async)
|   |           Sustained 1-per-clock throughput, buffered
|   |
|   +-- NO, occasional transfers -->
|       |
|       +-- Does the source need backpressure (flow control)?
|           |
|           +-- YES --> Closed-loop handshake
|           |           (cdc_2_phase_handshake or
cdc_4_phase_handshake)
|           |
|           +-- NO, source can guarantee hold time -->
|               |
|               +-- Open-loop (cdc_open_loop)
|                   Simplest, lowest area, no ack needed
|
Single-bit signal?
|
+-- Level / quasi-static --> cdc_synchronizer or glitch_free_n_dff_arn
+-- Single-cycle pulse   --> sync_pulse
+-- Reset signal          --> reset_sync
```

1.5.1 Quick Reference

Scenario	Module	Why
Config register update (CPU -> peripheral)	`cdc_open_loop`	Infrequent, no backpressure

Scenario	Module	Why
		needed
Interrupt signal (1 bit, pulse)	`sync_pulse`	Single-cycle event
Status register read (slow -> fast)	`cdc_synchronizer`	Quasi-static level
APB slave in different clock domain	`apb_slave_cdc`	Full protocol CDC
Monitor packet crossing	`cdc_2_phase_handshake`	Occasional, needs flow control
DMA data stream	`fifo_async`	High throughput, continuous
AXI channel crossing	`gaxi_skid_buffer_async`	Pipelined, GAXI protocol
Reset distribution	`reset_sync`	Async assert, sync deassert

1.6 Common Mistakes

1. Using a multi-flop synchronizer for multi-bit buses that change simultaneously

Multi-flop synchronizers (2FF/3FF) are safe for single-bit signals or Gray-coded values. If multiple bits can change at once, different bits may be sampled from different time points, producing a value that never existed.

Fix: Use a handshake or async FIFO.

2. Using set_false_path on CDC data buses

`set_false_path` tells the tool to ignore timing entirely. For handshake and open-loop CDC, the data bus must arrive within a bounded window (roughly SYNC_STAGES destination clocks). Use `set_max_delay -datapath_only` instead.

3. Assuming async FIFO depth = 2 is enough

A depth-2 async FIFO has only 1 usable entry (full when wr_ptr != rd_ptr by 1). The synchronized pointers lag by SYNC_STAGES cycles, so the effective throughput drops. Use depth >= 4 for practical designs.

4. Sending a new open-loop transfer before the previous one is sampled

Without an acknowledge, the source has no way to know the destination has latched the data. If the source writes again too soon, the first transfer is silently lost. Minimum spacing: SYNC_STAGES + 1 destination clocks.

1.7 References

- Clifford E. Cummings, “Clock Domain Crossing (CDC) Design and Verification Techniques Using SystemVerilog” (SNUG 2008)
- Clifford E. Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design” (SNUG 2002)
- IEEE 1800-2017 SystemVerilog LRM, Section on synchronizer modeling

---

Last Updated: 2026-04-21

1.8 Navigation

- Back to Shared Infrastructure Index
- Back to RTLAmba Index

2 CDC Synchronizer

Module: `cdc_synchronizer.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

2.1 Overview

Multi-bit synchronizer for clock domain crossing. Wraps the `glitch_free_n_dff_arn` primitive with a consistent naming interface for the AMBA infrastructure.

Suitable for quasi-static signals that change infrequently relative to the destination clock (configuration registers, status flags, level indicators). NOT suitable for pulse transfers or streaming data – use `sync_pulse` or async FIFOs for those.

2.1.1 Key Features

- Parameterizable bus width (1 to N bits)

- Configurable synchronizer depth (2-5 stages, default 3)
- Uses `glitch_free_n_dff_arn` internally for ASYNC_REG attributes
- Zero additional logic – pure flop chain

2.2 Module Interface

```
module cdc_synchronizer #(
    parameter int WIDTH      = 1,   // Bus width to synchronize
    parameter int FLOP_COUNT = 3    // Number of synchronizer stages
(2-5)
) (
    input  logic             clk,       // Destination clock
    input  logic             rst_n,     // Asynchronous active-low
reset
    input  logic [WIDTH-1:0] async_in,  // Asynchronous input from
source domain
    output logic [WIDTH-1:0] sync_out   // Synchronized output in
destination domain
);
```

2.3 Parameters

Parameter	Default	Range	Description
`WIDTH`	1	1+	Number of bits to synchronize
`FLOP_COUNT`	3	2-5	Synchronizer chain depth (stages)

2.4 Ports

Port	Direction	Width	Description
`clk`	input	1	Destination domain clock
`rst_n`	input	1	Asynchronous active-low reset
`async_in`	input	WIDTH	Input signal from source clock domain
`sync_out`	output	WIDTH	Synchronized output in destination domain

2.5 Usage Guidelines

- Input must be quasi-static: stable for at least `FLOP_COUNT + 1` destination clock cycles before being sampled
- For multi-bit buses, ALL bits must change simultaneously or use Gray coding
- Use `FLOP_COUNT=2` for relaxed MTBF, `FLOP_COUNT=3` for production (default)
- Do NOT use for pulse signals – use `sync_pulse` instead
- Do NOT use for streaming data – use `fifo_async` or `gaxi_fifo_async`

2.6 Instantiation Example

```
cdc_synchronizer #(
    .WIDTH      (4),
    .FLOP_COUNT (3)
) u_status_sync (
    .clk      (dest_clk),
    .rst_n    (dest_rst_n),
    .async_in (src_status),
    .sync_out (synced_status)
);
```

2.7 Resources

- RTL: `rtl/amba/shared/cdc_synchronizer.sv`
- Primitive: `rtl/common/glitch_free_n_dff_arn.sv`
- Formal: `formal/amba/cdc_synchronizer/`
- Related: `sync_pulse.sv` (pulse crossing), `cdc_2_phase_handshake.sv` (data + handshake)

3 Glitch-Free N-DFF Synchronizer (`glitch_free_n_dff_arn.sv`)

3.1 Purpose

Provides a parameterized multi-stage synchronizer for safe clock domain crossing (CDC). Reduces metastability risk through configurable stages of flip-flops while maintaining data integrity across asynchronous clock domains.

3.2 Ports

3.2.1 Input Ports

- **`clk`** - Destination domain clock
- **`rst_n`** - Destination domain active-low reset
- **`d[WIDTH-1:0]`** - Input data from source clock domain

3.2.2 Output Ports

- **`q[WIDTH-1:0]`** - Synchronized output data in destination domain

3.2.3 Parameters

- **`FLOP_COUNT`** - Number of synchronizer flip-flop stages (default: 3)
- **`WIDTH`** - Data width in bits (default: 4)

3.3 Clock Domain Crossing Theory

3.3.1 Metastability Problem

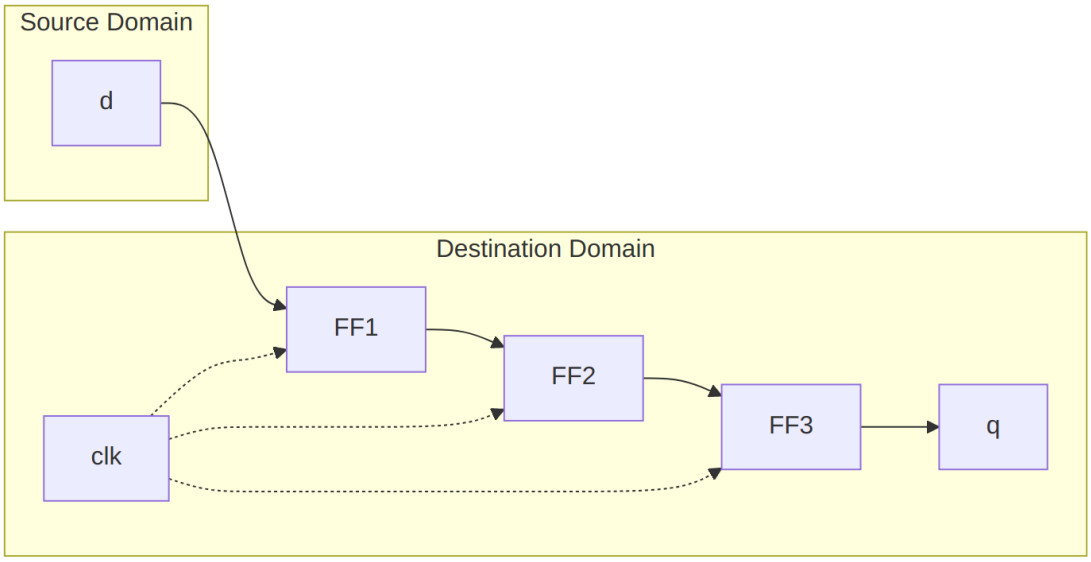
When signals cross between asynchronous clock domains:

```
Source Clock     __|‾|__|‾|__|‾|__|‾|__
Data Change      ______|_____________
Dest Clock       ___|‾|__|‾|__|‾|__|‾|_
Setup Violation?      ↑ Potential metastability
```

3.3.2 Metastability Effects

- **Undefined output**: Neither 0 nor 1, but intermediate voltage
- **Extended resolution**: Takes longer than normal clock period
- **Propagation**: Can corrupt downstream logic
- **System failure**: Can cause entire design malfunction

3.4 Multi-Stage Synchronizer Solution



Architecture Overview

3.4.1 Stage-by-Stage Analysis

1. **Stage 1 (FF1)**: Captures source data, may go metastable
2. **Stage 2 (FF2)**: Allows metastability to resolve, low probability of propagation
3. **Stage 3+ (FFN)**: Further reduces MTBF (Mean Time Between Failures)

3.5 Implementation Details

3.5.1 Packed Array Structure

```
logic [FC-1:0][WIDTH-1:0] r_q_array;
```

- **Efficient storage**: Single declaration for all stages
- **Clear indexing**: `r_q_array[0]` is first stage, `r_q_array[FC-1]` is last
- **Synthesis friendly**: Maps well to FPGA/ASIC resources

3.5.2 Shift Register Behavior

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // Reset all flip-flops to zero
        r_q_array <= {FC{{DW{1'b0}}}};
    end else begin
        // Load new data into first stage
```

```
        r_q_array[0] <= d;
        // Shift existing data through remaining stages
        for (int i = 1; i < FC; i++) begin
            r_q_array[i] <= r_q_array[i-1];
        end
    end
end
```

3.5.3 Reset Behavior

- **Synchronous reset**: Uses destination domain reset
- **Complete reset**: All stages reset to zero
- **Nested replication**: `{FC{{DW{1'b0}}}}` creates FC copies of DW zeros

3.6 MTBF (Mean Time Between Failures) Analysis

3.6.1 MTBF Calculation

```
MTBF = (e^(t_res/τ)) / (f_clk × f_data × τ)

Where:
- t_res = Resolution time available (clock period - setup - routing)
- τ = Flip-flop metastability time constant (~200ps typical)
- f_clk = Destination clock frequency
- f_data = Data transition frequency
```

3.6.2 Stage Count Impact

Stages	Relative MTBF	Typical MTBF
1	1× (baseline)	Minutes
2	~1000×	Hours
3	~1,000,000×	Years
4	~1,000,000,000×	Millennia

3.6.3 Practical Guidelines

- **2 stages**: Minimum for most applications
- **3 stages**: Recommended for high-reliability systems
- **4+ stages**: Extreme reliability requirements (aerospace, medical)

3.7 Latency vs. Reliability Trade-off

3.7.1 Synchronizer Latency

- **Minimum latency**: 2 clock cycles (2-stage synchronizer)
- **General latency**: `FLOP_COUNT` clock cycles
- **Fixed delay**: Independent of data pattern

3.7.2 Performance Impact

```
// 2-stage: 2 cycle latency, good MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(2)) fast_sync (...);

// 3-stage: 3 cycle latency, excellent MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(3)) safe_sync (...);

// 4-stage: 4 cycle latency, extreme MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(4)) ultra_safe_sync (...);
```

3.8 Usage Guidelines

3.8.1 Appropriate Use Cases

- **Control signals**: Reset, enable, mode switches
- **Slow status signals**: Error flags, configuration bits
- **Gray code pointers**: FIFO read/write pointers
- **Single-bit signals**: Generally safe for multi-bit if Gray coded

3.8.2 Inappropriate Use Cases

- **High-speed data buses**: Use proper CDC techniques (handshaking, FIFOs)
- **Binary counters**: Multiple bits can transition simultaneously
- **Address buses**: Binary addresses are not CDC-safe
- **Clock signals**: Never synchronize clocks this way

3.8.3 Multi-Bit Considerations

```
// SAFE: Gray-coded FIFO pointers
glitch_free_n_dff_arn #(.WIDTH(5)) gray_sync (
    .d(gray_pointer),      // Gray code - only 1 bit changes
    .q(sync_gray_pointer)
);

// UNSAFE: Binary counter synchronization
// Don't do this - multiple bits can change simultaneously
```

```
glitch_free_n_dff_arn #(.WIDTH(8)) bad_sync (
    .d(binary_counter),    // Binary - multiple bit transitions
    .q(sync_counter)       // May capture inconsistent values
);
```

3.9 Advanced Applications

3.9.1 Reset Synchronizer Pattern

```
// Asynchronous assert, synchronous deassert
logic reset_sync;
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(1)) reset_sync_inst (
    .clk(dest_clk),
    .rst_n(1'b1),          // No reset feedback
    .d(~async_reset),      // Invert for active-low
    .q(reset_sync)
);

assign sync_reset_n = reset_sync & ~async_reset;
```

3.9.2 Enable Signal Synchronization

```
glitch_free_n_dff_arn #(.FLOP_COUNT(3), .WIDTH(1)) enable_sync (
    .clk(fast_clk),
    .rst_n(rst_n),
    .d(slow_enable),
    .q(fast_enable_sync)
);
```

3.9.3 Status Flag Collection

```
// Synchronize multiple independent status signals
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(4)) status_sync (
    .clk(dest_clk),
    .rst_n(rst_n),
    .d({error_flag, done_flag, busy_flag, ready_flag}),
    .q({sync_error, sync_done, sync_busy, sync_ready})
);
```

3.10 Synthesis Considerations

3.10.1 FPGA Implementation

- **Dedicated resources**: Maps to flip-flop primitives
- **Placement**: Tools may place stages in different locations
- **Timing constraints**: Require proper CDC constraints

3.10.2 Timing Constraints

```
# Constrain the synchronizer input as asynchronous
set_false_path -to [get_pins sync_inst/r_q_array[0]/D]

# Or set maximum delay if there's some relationship
set_max_delay -to [get_pins sync_inst/r_q_array[0]/D] 2.0
```

3.10.3 Optimization Prevention

```
// Synthesis attributes to prevent optimization
(* ASYNC_REG = "TRUE" *) logic [FC-1:0][WIDTH-1:0] r_q_array;
(* DONT_TOUCH = "TRUE" *) logic [FC-1:0][WIDTH-1:0] r_q_array;
```

3.11 Verification Strategies

3.11.1 Functional Verification

- **Data integrity**: Verify output eventually matches input
- **Reset behavior**: Check proper reset operation
- **Latency verification**: Confirm expected delay

3.11.2 Metastability Testing

```
// Stress test with rapid data changes
initial begin
    forever begin
        @(posedge source_clk);
        d <= $random();
        #($random() % 100);  // Random timing
    end
end
```

3.11.3 Formal Verification

- **Eventually properties**: Data will eventually propagate
- **Reset properties**: Reset behavior verification
- **No X propagation**: Ensure no unknown values propagate

3.12 Debug Features

3.12.1 Waveform Analysis

```
// Flatten array for waveform viewing
wire [(DW*FC)-1:0] flat_r_q;
genvar i;
generate
    for (i = 0; i < FC; i++) begin : gen_flatten_memory
```

```
        assign flat_r_q[i*DW+:DW] = r_q_array[i];
    end
endgenerate
```

3.12.2 Simulation Visibility

- **All stages visible**: Can observe data propagation through stages
- **Metastability injection**: Can inject X values for testing
- **Timing analysis**: Observe setup/hold violations

3.13 Related Modules

- **FIFO CDC modules**: Use this for pointer synchronization
- **Reset synchronizers**: Specialized CDC for reset signals
- **Handshake synchronizers**: For complex data CDC
- **Pulse synchronizers**: For single-cycle pulse CDC

3.14 Navigation

- **← Back to RTLCommon Index**
- **← Back to Main Documentation Index**

4 CDC Open-Loop

Module: `cdc_open_loop.sv` **Location:** `rtl/amba/shared/` **Category:** Open-loop CDC (see CDC Primer)

4.1 Overview

Open-loop multi-bit CDC using valid/data stretching. The source captures data on a single-cycle `src_valid` pulse and holds both data and a stretched valid level for `STRETCH_CYCLES` source clock cycles. The destination synchronizes the stretched valid and latches data on its rising edge.

No return path or handshake exists. The source is busy for exactly `STRETCH_CYCLES` source clocks, then free to send again. The stretch length can either be set manually (`STRETCH_CYCLES`) or computed from the source/destination clock frequencies (`AUTO_STRETCH=1` with `SRC_CLK_HZ` and `DST_CLK_HZ`).

4.1.1 Key Features

- True open-loop: no return path, no handshake, no backpressure

- Source holds data and stretched valid for a parameterizable number of source clocks
- Destination uses multi-stage synchronizer + rising edge detect to latch data
- `src_busy` blocks the next transfer during the stretch window
- Optional auto-compute: pass clock frequencies and the module sizes `STRETCH` itself
- Minimal area: one counter, one synchronizer, one edge detector

4.1.2 When to Use

- Clock ratio is known and fixed (or bounded)
- Destination is always ready (no flow control needed)
- Configuration register updates, status flags, infrequent events
- Minimum area/complexity is desired

4.1.3 When NOT to Use

- Clock ratio unknown or variable – use `cdc_2_phase_handshake` (closed-loop)
- Destination needs backpressure – use a handshake
- Streaming data – use `fifo_async` or `gaxi_fifo_async`

4.2 Module Interface

```
module cdc_open_loop #(
    parameter int DATA_WIDTH     = 8,
    parameter int STRETCH_CYCLES = 8,
    parameter int SYNC_STAGES    = 2,

    // Optional auto-compute of STRETCH_CYCLES from clock frequencies
    parameter int AUTO_STRETCH = 0,             // 0 = use
STRETCH_CYCLES; 1 = auto
    parameter int SRC_CLK_HZ   = 25_000_000,    // Fastest source
clock to handle
    parameter int DST_CLK_HZ   = 100_000_000    // Destination clock
frequency
) (
    // Source clock domain
    input  logic                  clk_src,
    input  logic                  rst_src_n,
    input  logic                  src_valid,     // Single-cycle pulse
    input  logic [DATA_WIDTH-1:0] src_data,      // Data to transfer
    output logic                  src_busy,      // High during
```

```
stretch

    // Destination clock domain
    input  logic                  clk_dst,
    input  logic                  rst_dst_n,
    output logic                  dst_valid,     // Single-cycle pulse
    output logic [DATA_WIDTH-1:0] dst_data       // Latched data
);
```

4.3 Parameters

Parameter	Default	Description
DATA_WIDTH	8	Width of the data bus
STRETCH_CYCLES	8	Source clocks to hold valid+data stable (used when AUTO_STRETCH=0)
SYNC_STAGES	2	Destination synchronizer depth (2-4; raise for extra MTBF margin)
AUTO_STRETCH	0	0 = use STRETCH_CYCLES; 1 = compute from SRC_CLK_HZ / DST_CLK_HZ
SRC_CLK_HZ	25_000_000	Source clock frequency (used when AUTO_STRETCH=1)
DST_CLK_HZ	100_000_000	Destination clock frequency (used when AUTO_STRETCH=1)

4.3.1 Setting STRETCH_CYCLES manually (AUTO_STRETCH=0)

This is a multi-bit data CDC, so the source data bus must remain stable until the synchronized enable has propagated through the destination sync chain *and* loaded the destination register. That gives:

```
STRETCH_CYCLES >= ceil((SYNC_STAGES + 1) * T_dst / T_src)
```

`(SYNC_STAGES + 1)` counts the synchronizer FFs plus the destination load edge. The 1.5 × T_dst "rule of thumb" you may have seen is enough to capture the valid pulse alone, but a multi-bit datapath needs the longer hold so `r_src_data` is still stable when the destination latches it. See *References* below.

Source Clock	Dest Clock	Ratio	SYNC_STAGES	Min STRETCH_CYCLES
100 MHz	100 MHz	1:1	2	3
100 MHz	50 MHz	2:1	2	6
100 MHz	25 MHz	4:1	2	12
1 GHz	100 MHz	10:1	2	30
200 MHz	25 MHz	8:1	2	24

When in doubt, round up — extra stretch cycles cost only source-side latency, not area.

4.3.2 Auto-computed STRETCH (AUTO_STRETCH=1)

Set AUTO_STRETCH=1 and pass the source/destination clock frequencies in Hz. The module computes STRETCH_CYCLES at elaboration time using:

```
STRETCH = ceil((SYNC_STAGES + 1) * SRC_CLK_HZ / DST_CLK_HZ)
```

The result is a localparam — folds to a literal in synthesis, no runtime cost. SRC_CLK_HZ should be the **fastest** source clock you intend to support; sources running faster will start dropping pulses.

When AUTO_STRETCH=1 the STRETCH_CYCLES parameter is ignored. When AUTO_STRETCH=0 (default) the auto-formula is dead code and STRETCH_CYCLES is used as-is, so existing callers see no change.

Source SRC_CLK_HZ	Destination DST_CLK_HZ	Computed STRETCH (SYNC=2)
25 MHz	100 MHz	1
50 MHz	100 MHz	2
100 MHz	100 MHz	3
100 MHz	40 MHz	8
100 MHz	25 MHz	12
1 GHz	100 MHz	30

Prefer auto when the clock plan is captured in a single top-level location — propagate SRC_CLK_HZ / DST_CLK_HZ as parameters from the top and every instance retunes itself if the clock plan changes.

4.3.3 Back-to-back pulses

src_busy only guards the HIGH stretch window. After it falls, the destination sync chain still needs roughly (SYNC_STAGES + 1) * T_dst to see the falling edge before another rising edge

can be detected. If the source asserts the next `src_valid` immediately after `src_busy` falls, two HIGH stretches can merge in the dst view and the second pulse is silently dropped.

Two ways to keep this safe:

1. Pick `STRETCH_CYCLES` large enough that one full source-side LOW cycle is comfortably more than `(SYNC_STAGES + 1) * T_dst`. (For slow-source → fast-dst this is automatic; for fast-source → slow-dst you may need extra margin.)
2. Have the source pace pulses: wait `src_busy` to fall, then idle for `ceil((SYNC_STAGES + 1) * T_dst / T_src) + 1` source cycles before the next `src_valid`.

The verification testbench (`bin/TBClasses/amba/cdc_open_loop.py`) enforces option 2 in its no-loss phases.

4.4 Ports

Port	Direction	Width	Description
`clk_src`	input	1	Source domain clock
`rst_src_n`	input	1	Source async reset (active low)
`src_valid`	input	1	Single-cycle pulse: data is valid
`src_data`	input	DATA_WIDTH	Data to transfer
`src_busy`	output	1	High during stretch countdown
`clk_dst`	input	1	Destination domain clock
`rst_dst_n`	input	1	Destination async reset (active low)
`dst_valid`	output	1	Single-cycle pulse: data latched
`dst_data`	output	DATA_WIDTH	Latched data (stable until next dst_valid)

4.5 Architecture

```
Source Domain (clk_src):                Destination Domain (clk_dst):

src_valid --+
            v
src_data --> [r_src_data] ---------------------------> [r_dst_data] --
> dst_data
             (captured)         (quasi-static)         (latch on
rise)

src_valid --> [r_src_valid_stretch] --> [SYNC] --> [edge detect] -->
dst_valid
              held high for                 glitch_free
              STRETCH_CYCLES                n_dff_arn
                  |
            [r_stretch_count]
            countdown to 0
                  |
            src_busy = (count != 0)
```

4.5.1 Timing Diagram

```
src_clk:     |~|_|~|_|~|_|~|_|~|_|~|_|~|_|~|_|~|_|~|_|~|_|~|_|
src_valid:   ____|~~|_____________________________________________
src_data:    ====|AAAA|=========================================
src_busy:    ________|~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~|_______
r_stretch:   ________|~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~|_______
                      <-- STRETCH_CYCLES src clocks -->

dst_clk:     |~~~|___|~~~|___|~~~|___|~~~|___|~~~|___|~~~|___|
w_valid_sync:________________________|~~~~~~~~~~~~|___________
dst_valid:   ____________________________________|~~|___________
dst_data:    ================================|AAAA|==========
```

4.6 Usage Example

4.6.1 Manual STRETCH (back-compat default)

```
// CPU writes config register at 100 MHz, peripheral runs at 25 MHz
// Ratio = 4:1, STRETCH_CYCLES = ceil((2+1) * 4) = 12
cdc_open_loop #(
    .DATA_WIDTH     (32),
    .STRETCH_CYCLES (12),
    .SYNC_STAGES    (2)
) u_cfg_cdc (
```

```
    .clk_src   (cpu_clk),        // 100 MHz
    .rst_src_n (cpu_rst_n),
    .src_valid (cfg_write_en),
    .src_data  (cfg_write_data),
    .src_busy  (cfg_busy),       // Block next write

    .clk_dst   (periph_clk),    // 25 MHz
    .rst_dst_n (periph_rst_n),
    .dst_valid (cfg_update),
    .dst_data  (cfg_value)
);
```

4.6.2 Auto STRETCH from clock frequencies

```
// Same CPU→peripheral CDC, but let the module size STRETCH itself.
// If the clock plan changes (e.g. CPU bumped to 200 MHz) you only
// update SRC_CLK_HZ at the top and every instance retunes.
localparam int CPU_HZ    = 100_000_000;
localparam int PERIPH_HZ = 25_000_000;

cdc_open_loop #(
    .DATA_WIDTH   (32),
    .SYNC_STAGES  (2),
    .AUTO_STRETCH (1),
    .SRC_CLK_HZ   (CPU_HZ),
    .DST_CLK_HZ   (PERIPH_HZ)
) u_cfg_cdc (
    .clk_src   (cpu_clk),
    .rst_src_n (cpu_rst_n),
    .src_valid (cfg_write_en),
    .src_data  (cfg_write_data),
    .src_busy  (cfg_busy),

    .clk_dst   (periph_clk),
    .rst_dst_n (periph_rst_n),
    .dst_valid (cfg_update),
    .dst_data  (cfg_value)
);
```

4.7 SDC Constraints

```
# 1. Stretched valid (single-bit level, synchronized by destination)
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_valid_stretch_reg/C] \
    -to   [get_pins u_cdc/u_valid_sync/q_reg[0]/D] \
    <dst_period_ns>
```

```
# 2. Data bus (quasi-static, held for STRETCH_CYCLES src clocks)
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_data_reg[*]/C] \
    -to   [get_pins u_cdc/r_dst_data_reg[*]/D] \
    <dst_period_ns>
```

Do NOT use `set_false_path` – the data bus must arrive within a bounded window.

4.8 Resources

- RTL: `rtl/amba/shared/cdc_open_loop.sv`
- Depends on: `rtl/common/glitch_free_n_dff_arn.sv`
- Testbench: `bin/TBClasses/amba/cdc_open_loop.py`
- Test runner: `val/amba/test_cdc_open_loop.py` (28 configs covering manual, auto, and cliff)
- CDC Primer: cdc_primer.md
- Closed-loop alternative: cdc_2_phase_handshake.md

4.9 References

The `(SYNC_STAGES + 1) × T_dst` data-hold requirement follows directly from the MCP (Multi-Cycle Path) formulation: the source data must stay stable until the synchronized enable has traversed the destination sync chain and loaded the destination register. The exact closed-form formula is this module's own — the cited references state the principle qualitatively and leave the cycle count to the reader.

- Clifford E. Cummings, *Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog*, SNUG Boston 2008 — original MCP formulation.
- Verilog Pro, *Clock Domain Crossing Design — Part 2*: "the input data has to be held until the synchronization pulse loads the data in the destination clock domain. The whole process takes at least two destination clocks." (= `SYNC_STAGES + 1` destination cycles for a 2-FF synchronizer.)
- thedatabus.in, *Clock Domain Crossing (CDC): The Complete Reference Guide*: "ensure that the source holds the information long enough for it to be safely taken through all the synchronizer stages."
- EDN, *Understanding Clock Domain Crossing*.

Last Updated: 2026-06-26

4.10 Navigation

- Back to CDC Primer
- Back to Shared Infrastructure Index

5 CDC 2-Phase Handshake

Module: `cdc_2_phase_handshake.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready
Companion: `cdc_4_phase_handshake.sv` (level-based, classic variant)

5.1 Overview

Two-phase (toggle / NRZ) CDC handshake with data transfer. Provides the same valid/ready interface as the 4-phase variant but uses toggle-based signaling to halve the control-path round trip – two synchronizer crossings per transfer instead of four.

The "event" is a TRANSITION of the request/acknowledge signals rather than a level. The source toggles its request bit on each new transfer; the destination detects the toggle edge and responds by toggling its acknowledge bit.

5.1.1 Key Features

- Drop-in replacement for `cdc_4_phase_handshake` (same port interface)
- Faster: 2 synchronizer crossings per transfer (vs 4 for level-based)
- Toggle-based request/acknowledge (NRZ encoding)
- Parameterizable synchronizer depth (2-3 stages)
- Optional timeout detection for stall diagnosis
- ASYNC_REG attributes for Xilinx/Intel FPGA flows
- SDC constraint guidance in RTL comments

5.1.2 2-Phase vs 4-Phase Comparison

Aspect	2-Phase (this module)	4-Phase
Crossings per transfer	2 (req toggle + ack toggle)	4 (req + ack + req clear + ack clear)
Latency	~5-6 destination clocks	~7-8 destination

Aspect	2-Phase (this module)	4-Phase
		clocks
Throughput	Higher (shorter round-trip)	Lower
Complexity	Edge detection required	Simpler level-based logic
Reset behavior	Toggle state preserved across reset recovery	Clean level-based reset

5.2 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	8	Width of the data bus (1+)
SYNC_STAGES	int	3	Synchronizer depth for req/ack (2 or 3)
TIMEOUT_CYCLES	int	0	0 = disabled; >0 asserts src_timeout after stall

5.3 Port Groups

5.3.1 Source Clock Domain Interface

Port	Direction	Width	Description
clk_src	input	1	Source domain clock
rst_src_n	input	1	Source domain active-low async reset
src_valid	input	1	Source indicates data valid
src_ready	output	1	Handshake ready (asserted when idle)
src_data	input	DATA_WIDTH	Data from

Port	Direction	Width	Description
			source domain
src_timeout	output	1	Stall timeout (when TIMEOUT_CYCLES > 0)

5.3.2 Destination Clock Domain Interface

Port	Direction	Width	Description
clk_dst	input	1	Destination domain clock
rst_dst_n	input	1	Destination domain active-low async reset
dst_valid	output	1	Data valid to receiver
dst_ready	input	1	Receiver ready
dst_data	output	DATA_WIDTH	Data transferred to destination domain

5.4 Theory of Operation

5.4.1 Source Domain FSM

```
S_IDLE      src_ready=1. On src_valid: latch data, toggle r_req_tog,
            drop src_ready, go to S_WAIT_ACK.

S_WAIT_ACK  Wait for ack edge (w_ack_event). On edge: src_ready=1,
            return to S_IDLE.
```

5.4.2 Destination Domain FSM

```
D_IDLE       Wait for req edge (w_req_event). On edge: copy data into
             r_dst_data, raise dst_valid, go to D_WAIT_READY.

D_WAIT_READY Hold dst_valid. On dst_ready: drop dst_valid, toggle
             r_ack_tog, return to D_IDLE.
```

5.4.3 Edge Detection

```
event = current_sync_output ^ previous_sync_output
```

The edge detector uses one additional flop per direction to compare the current synchronized toggle value with the previous cycle's value. A difference indicates a new transfer event.

5.5 Usage Example

```
cdc_2_phase_handshake #(
    .DATA_WIDTH     (64),
    .SYNC_STAGES    (3),
    .TIMEOUT_CYCLES (1000)
) u_packet_cdc (
    // Source: Monitor clock domain
    .clk_src     (mon_clk),
    .rst_src_n   (mon_rst_n),
    .src_valid   (mon_pkt_valid),
    .src_ready   (mon_pkt_ready),
    .src_data    (mon_pkt_data),
    .src_timeout (mon_timeout),

    // Destination: System clock domain
    .clk_dst     (sys_clk),
    .rst_dst_n   (sys_rst_n),
    .dst_valid   (sys_pkt_valid),
    .dst_ready   (sys_pkt_ready),
    .dst_data    (sys_pkt_data)
);
```

5.5.1 Swapping with 4-Phase

The two handshake modules are drop-in interchangeable:

```
// Change only the module name -- ports are identical
cdc_4_phase_handshake #(  // or cdc_2_phase_handshake
    .DATA_WIDTH     (64),
    .SYNC_STAGES    (3),
    .TIMEOUT_CYCLES (0)
) u_cdc ( ... );
```

5.6 Required SDC Constraints

```
# 1. Req / Ack single-bit toggle crossings
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_req_tog_reg/C] \
    -to   [get_pins u_cdc/r_req_sync_reg[0]/D] \
    <dst_period_ns>

set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_ack_tog_reg/C] \
```

```
    -to   [get_pins u_cdc/r_ack_sync_reg[0]/D] \
    <src_period_ns>

# 2. Data bus (quasi-static, protected by toggle handshake)
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_data_hold_reg[*]/C] \
    -to   [get_pins u_cdc/r_dst_data_reg[*]/D] \
    <dst_period_ns>
```

Do NOT use `set_false_path` – the data bus relies on a bounded crossing window for correctness.

5.7 Related Modules

Module	Purpose
`cdc_4_phase_handshake.sv`	Level-based variant (slower, simpler)
`cdc_synchronizer.sv`	Multi-bit sync (no handshake, quasi-static only)
`sync_pulse.sv`	Single-cycle pulse crossing (no data)
`fifo_async.sv`	Streaming data crossing (Gray-coded pointers)
`apb_slave_cdc.sv`	APB protocol CDC (uses either handshake variant)

5.8 Resources

- RTL: `rtl/amba/shared/cdc_2_phase_handshake.sv`
- Tests: `val/amba/test_cdc_2_phase_handshake.py`
- Formal: `formal/amba/cdc_handshake/`

Last Updated: 2026-04-21

5.9 Navigation

- Back to Shared Infrastructure Index
- Back to RTLAmba Index

6 CDC 4-Phase Handshake

Module: `cdc_4_phase_handshake.sv` (renamed from `cdc_4_phase_handshake.sv`) **Location:** `rtl/amba/shared/` **Status:** Production Ready **Companion:** `cdc_2_phase_handshake.sv` (toggle-based, faster variant)

6.1 Overview

The CDC Handshake module provides robust clock domain crossing (CDC) with valid/ready flow control protocol. It implements a dual FSM architecture with 3-stage synchronizers to safely transfer data between asynchronous clock domains while maintaining AXI-style handshaking semantics.

6.1.1 Key Features

- Safe asynchronous clock domain crossing with metastability protection
- AXI-compatible valid/ready handshake protocol
- 3-stage synchronizer chains for REQ/ACK signals
- Dual FSM architecture (source and destination domains)
- Parameterizable data width
- Independent reset for each clock domain
- Proven REQ/ACK protocol for glitch-free operation

6.2 Module Purpose

Modern SoC designs frequently require data transfer between asynchronous clock domains. Direct signal crossing risks metastability, data corruption, and protocol violations. The CDC Handshake module solves these challenges by:

1. **Metastability Protection:** Multi-stage synchronizers for control signals
2. **Data Integrity:** Stable data capture with proper handshaking
3. **Flow Control:** Backpressure support via ready signals
4. **Protocol Correctness:** AXI-compatible valid/ready semantics

Use Cases: - AXI interface clock domain crossing (monitor packets, control signals) - Data transfer between processor cores and peripherals - Multi-clock SoC interconnect boundaries - Configuration register updates across clock domains

Key Guarantee: Data transferred without corruption or loss, with proper flow control.

6.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	8	Width of the data bus (1 to 1024+ bits typical)
SYNC_STAGES	int	3	Synchronizer depth for req/ack (2 or 3)
TIMEOUT_CYCLES	int	0	0 = disabled; >0 asserts src_timeout after stall
FAST_PATH	bit	0	1 = destination fast-path when dst_ready already high

Typical Values: - Control signals: 8-32 bits - AXI monitor packets: 64 bits - Configuration registers: 32-64 bits - Custom data structures: Application-specific

6.4 Port Groups

6.4.1 Source Clock Domain Interface

Port	Direction	Width	Description
clk_src	input	1	Source domain clock (can be asynchronous to clk_dst)
rst_src_n	input	1	Source domain active-low asynchronous reset
src_valid	input	1	Source indicates data valid (AXI-style valid)
src_ready	output	1	Handshake ready back to source (asserted when transfer complete)
src_data	input	DATA_WIDTH	Data from source domain (captured when valid asserted)

6.4.2 Destination Clock Domain Interface

Port	Direction	Width	Description
clk_dst	input	1	Destination domain clock (can be asynchronous to clk_src)
rst_dst_n	input	1	Destination domain active-low asynchronous reset
dst_valid	output	1	Destination indicates data valid to receiver (AXI-style valid)
dst_ready	input	1	Receiver ready in destination domain (AXI-style ready)
dst_data	output	DATA_WIDTH	Data transferred to destination domain (stable when valid)

6.5 Functional Description

6.5.1 CDC Protocol Overview

The module implements a four-phase handshake protocol:

1. **Request Phase:** Source asserts REQ, latches data
2. **Synchronization Phase:** REQ crosses to destination domain (3 stages)
3. **Acknowledge Phase:** Destination asserts ACK after accepting data
4. **Synchronization Phase:** ACK crosses back to source domain (3 stages)
5. **Completion Phase:** Both sides deassert signals, ready for next transfer

Critical Property: Data stable throughout handshake (captured at REQ assertion, held until ACK completion).

6.5.2 Source Domain FSM

State Diagram:

```
S_IDLE → S_WAIT_ACK → S_WAIT_ACK_CLR → S_IDLE
```

State Descriptions:

S_IDLE: - Awaits src_valid assertion - src_ready = 1 (ready for new data) - r_req_src = 0 (request inactive)

S_WAIT_ACK: - Transfer in progress - src_ready = 0 (busy) - r_req_src = 1 (request asserted) - Data latched in r_async_data - Awaits synchronized ACK from destination

S_WAIT_ACK_CLR: - ACK received, completing handshake - src_ready = 0 (still busy) - r_req_src = 0 (request deasserted) - Awaits ACK to clear (return to 0)

State Transitions:

```
S_IDLE:
    if (src_valid):
        r_async_data <= src_data       // Capture data
        r_req_src <= 1                 // Assert request
        src_ready <= 0
        → S_WAIT_ACK

S_WAIT_ACK:
    if (w_ack_sync):                   // ACK received
        r_req_src <= 0                 // Deassert request
        → S_WAIT_ACK_CLR
    else:
        remain in S_WAIT_ACK

S_WAIT_ACK_CLR:
    if (!w_ack_sync):                  // ACK cleared
        src_ready <= 1                 // Ready for next transfer
        → S_IDLE
    else:
        remain in S_WAIT_ACK_CLR
```

6.5.3 Destination Domain FSM

State Diagram:

```
D_IDLE → D_WAIT_READY → D_WAIT_REQ_CLR → D_IDLE
```

State Descriptions:

D_IDLE: - Awaits synchronized REQ from source - dst_valid = 0 (no data available) - r_ack_dst = 0 (acknowledge inactive)

D_WAIT_READY: - REQ detected, data available - dst_valid = 1 (presenting data to receiver) - r_ack_dst = 0 (waiting for receiver acceptance) - Awaits dst_ready assertion

D_WAIT_REQ_CLR: - Data accepted, handshake completing - dst_valid = 0 (transfer complete) - r_ack_dst = 1 (acknowledge asserted) - Awaits REQ to clear (return to 0)

State Transitions:

```
D_IDLE:
    if (w_req_sync):                    // REQ received
        r_dst_data <= r_async_data      // Latch data
        dst_valid <= 1                  // Present to receiver
        → D_WAIT_READY

D_WAIT_READY:
    if (dst_ready):                     // Receiver accepts data
        r_ack_dst <= 1                  // Assert acknowledge
        dst_valid <= 0                  // Clear valid
        → D_WAIT_REQ_CLR
    else if (!w_req_sync):              // REQ withdrawn (error case)
        dst_valid <= 0
        → D_IDLE

D_WAIT_REQ_CLR:
    if (!w_req_sync):                   // REQ cleared
        r_ack_dst <= 0                  // Deassert acknowledge
        → D_IDLE
    else:
        remain in D_WAIT_REQ_CLR
```

6.5.4 Synchronizer Architecture

3-Stage Synchronizers for Metastability Protection:

REQ Synchronizer (Source → Destination):

```
always_ff @(posedge clk_dst or negedge rst_dst_n) begin
    if (!rst_dst_n)
        r_req_sync <= 3'b000;
    else
        r_req_sync <= {r_req_sync[1:0], r_req_src};
end

assign w_req_sync = r_req_sync[2];  // Use final stage
```

ACK Synchronizer (Destination → Source):

```
always_ff @(posedge clk_src or negedge rst_src_n) begin
    if (!rst_src_n)
        r_ack_sync <= 3'b000;
    else
        r_ack_sync <= {r_ack_sync[1:0], r_ack_dst};
end

assign w_ack_sync = r_ack_sync[2];  // Use final stage
```

Why 3 Stages? - Stage 1: May go metastable (timing violation from async input) - Stage 2: Metastability resolves (high probability) - Stage 3: Clean synchronized signal (used by FSM)

MTBF Calculation:

```
MTBF = e^(Tr/τ) / (f_clk × f_data × τ)

Where:
- Tr: Resolution time (2 clock periods for 2-stage sync)
- τ: Metastability time constant (technology-dependent, ~100ps)
- f_clk: Destination clock frequency
- f_data: Data change frequency

3-stage sync provides astronomical MTBF (years to millennia)
```

6.5.5 Data Stability Guarantee

Critical Timing:

1. **Data Capture (Source Domain):**
   - Data captured when src_valid asserted in S_IDLE
   - Stored in r_async_data register
   - Held stable until handshake completes
2. **Data Transfer (Asynchronous):**
   - r_async_data crosses clock domains (static signal)
   - No timing violations (data stable throughout)
3. **Data Latch (Destination Domain):**
   - Data latched into r_dst_data when REQ synchronized
   - Held stable until dst_ready asserted

Key Property: Data never changes while crossing domains (setup/hold violations impossible).

6.6 Usage Example

6.6.1 Basic CDC for Monitor Packets (64-bit)

```
// Monitor packets generated in monitor clock domain
// Must cross to system clock domain for processing

cdc_4_phase_handshake #(
    .DATA_WIDTH         (64)  // AXI monitor packet width
) u_packet_cdc (
    // Source: Monitor clock domain
```

```
    .clk_src            (mon_clk),
    .rst_src_n          (mon_rst_n),
    .src_valid          (mon_pkt_valid),
    .src_ready          (mon_pkt_ready),
    .src_data           (mon_pkt_data),

    // Destination: System clock domain
    .clk_dst            (sys_clk),
    .rst_dst_n          (sys_rst_n),
    .dst_valid          (sys_pkt_valid),
    .dst_ready          (sys_pkt_ready),
    .dst_data           (sys_pkt_data)
);

// Source domain: Monitor packet generation
always_ff @(posedge mon_clk or negedge mon_rst_n) begin
    if (!mon_rst_n) begin
        mon_pkt_valid <= 1'b0;
    end else begin
        if (packet_generated && mon_pkt_ready) begin
            mon_pkt_valid <= 1'b1;
            mon_pkt_data <= new_packet;
        end else if (mon_pkt_ready) begin
            mon_pkt_valid <= 1'b0;
        end
    end
end

// Destination domain: Packet processing
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        sys_pkt_ready <= 1'b0;
    end else begin
        if (sys_pkt_valid && processing_ready) begin
            sys_pkt_ready <= 1'b1;
            process_packet(sys_pkt_data);
        end else begin
            sys_pkt_ready <= 1'b0;
        end
    end
end
```

6.6.2 Configuration Register Update Across Domains

```
// CPU writes config register in AHB clock domain
// Design uses config in AXI clock domain

cdc_4_phase_handshake #(
```

```
    .DATA_WIDTH         (32)  // Configuration register width
) u_config_cdc (
    // Source: CPU/AHB clock domain
    .clk_src            (ahb_clk),
    .rst_src_n          (ahb_rst_n),
    .src_valid          (cfg_write_valid),
    .src_ready          (cfg_write_ready),
    .src_data           (cfg_write_data),

    // Destination: AXI logic clock domain
    .clk_dst            (axi_clk),
    .rst_dst_n          (axi_rst_n),
    .dst_valid          (cfg_update_valid),
    .dst_ready          (cfg_update_ready),
    .dst_data           (cfg_update_data)
);

// Source: AHB write decoder
always_ff @(posedge ahb_clk) begin
    if (ahb_write && addr_match && cfg_write_ready) begin
        cfg_write_valid <= 1'b1;
        cfg_write_data <= ahb_wdata;
    end else if (cfg_write_ready) begin
        cfg_write_valid <= 1'b0;
    end
end

// Destination: Configuration register update
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        config_reg <= 32'h0;
        cfg_update_ready <= 1'b0;
    end else begin
        if (cfg_update_valid) begin
            config_reg <= cfg_update_data;
            cfg_update_ready <= 1'b1;
        end else begin
            cfg_update_ready <= 1'b0;
        end
    end
end
```

6.6.3 Burst Transfer with Backpressure

```
// Transfer burst of data with flow control

cdc_4_phase_handshake #(
    .DATA_WIDTH         (128)  // Large data path
```

```
) u_burst_cdc (
    .clk_src            (src_clk),
    .rst_src_n          (src_rst_n),
    .src_valid          (burst_valid),
    .src_ready          (burst_ready),
    .src_data           (burst_data),

    .clk_dst            (dst_clk),
    .rst_dst_n          (dst_rst_n),
    .dst_valid          (fifo_wr_valid),
    .dst_ready          (fifo_wr_ready),
    .dst_data           (fifo_wr_data)
);

// Source: Burst generator
logic [7:0] burst_count;

always_ff @(posedge src_clk or negedge src_rst_n) begin
    if (!src_rst_n) begin
        burst_count <= 8'd0;
        burst_valid <= 1'b0;
    end else begin
        if (start_burst) begin
            burst_count <= burst_length;
            burst_valid <= 1'b1;
            burst_data <= first_data;
        end else if (burst_valid && burst_ready) begin
            if (burst_count > 1) begin
                burst_count <= burst_count - 1;
                burst_data <= next_data;
                burst_valid <= 1'b1;
            end else begin
                burst_valid <= 1'b0;
            end
        end
    end
end

// Destination: FIFO with backpressure
assign fifo_wr_ready = !fifo_full;
```

6.7 Design Notes

6.7.1 Handshake Protocol Timing

Latency Characteristics:

Minimum Transfer Latency (Fast Case): 1. Source asserts REQ (clk_src cycle N) 2. REQ synchronization: 3 clk_dst cycles 3. Destination asserts dst_valid (clk_dst cycle N+3) 4. Destination samples dst_ready (clk_dst cycle N+3 or later) 5. Destination asserts ACK (clk_dst cycle N+4) 6. ACK synchronization: 3 clk_src cycles 7. Source sees ACK (clk_src cycle N+7) 8. Source ready for next transfer (clk_src cycle N+8)

Total: 6-8 cycles in fast clock + 3-4 cycles in slow clock

Maximum Transfer Latency (Slow Case): - Add dst_ready assertion delay (unbounded, depends on receiver) - Add clock frequency ratio effects

Throughput: - Limited by handshake round-trip latency - Not suitable for high-throughput streaming (use async FIFO instead) - Optimal for control signals, configuration, low-rate data

6.7.2 Reset Considerations

Independent Domain Resets: - Each FSM has domain-specific reset - Resets can be asynchronous to each other - Design handles reset in one domain while other operates

Reset Recovery: 1. Source domain reset: FSM returns to S_IDLE, REQ=0 2. Destination domain reset: FSM returns to D_IDLE, ACK=0 3. Any in-flight handshake abandoned (safe by design) 4. Synchronizers clear to 3'b000 (deasserted state)

Recommendation: Assert both resets together during system initialization for clean startup.

6.7.3 Data Width Considerations

Wide Data Buses (64+ bits): - Large fanout from r_async_data register - Consider pipelining if timing closure issues - Area cost: DATA_WIDTH × 2 flip-flops (r_async_data + r_dst_data)

Narrow Data Buses (1-8 bits): - Minimal area overhead - Still requires full handshake protocol (latency same as wide buses)

Alternative for Streaming: If high-throughput needed, use async FIFO instead.

6.7.4 Metastability Protection Validation

Synchronizer Verification:

Static Timing Analysis: - Ensure no timing paths between asynchronous domains (except through synchronizers) - Verify synchronizer stages have no combinational logic - Check recovery/removal timing on first stage (expect violations, safe by design)

Formal Verification:

```
// CDC assertion: REQ/ACK only change in respective domains
assert property (@(posedge clk_src)
    !$rose(r_ack_dst) && !$fell(r_ack_dst));  // ACK only changes in
dst domain

assert property (@(posedge clk_dst)
    !$rose(r_req_src) && !$fell(r_req_src));  // REQ only changes in
src domain
```

Simulation: - Use formal CDC checkers (Synopsys SpyGlass, Cadence JasperGold) - Verify no X-propagation in synchronizer stages

6.7.5 Performance Optimization

When to Use CDC Handshake: - Control signals (low rate, <1% of clock frequency) - Configuration registers (infrequent updates) - Status synchronization (occasional transfers)

When to Use Async FIFO Instead: - High-throughput data streaming - Continuous data flow - Burst transfers with sustained rate

Hybrid Approach:

```
// Use async FIFO for data, CDC handshake for control
async_fifo #(.WIDTH(64), .DEPTH(16)) u_data_fifo (...);
cdc_4_phase_handshake #(.WIDTH(8)) u_ctrl_cdc (...);
```

6.8 Related Modules

6.8.1 Used By

- AXI monitor bus aggregation (packet crossing between monitor and system clocks)
- Configuration register synchronization
- Status/control signal crossing in multi-clock systems
- Peripheral interface adapters with asynchronous clocks

6.8.2 Dependencies

- **reset_defs.svh** - Standard reset macro definitions

6.8.3 See Also

- **async_fifo** - High-throughput async FIFO for streaming data
- **reset_sync.sv** - Reset synchronizer for single-bit reset crossing

---

6.9 References

6.9.1 Source Code

- RTL: `rtl/amba/shared/cdc_4_phase_handshake.sv`
- Tests: `val/amba/test_cdc_4_phase_handshake.py`

6.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
- CDC Best Practices: Clifford E. Cummings, “Clock Domain Crossing (CDC) Design & Verification Techniques”
- Design Guide: `rtl/amba/PRD.md`

6.9.3 Industry Standards

- IEEE 1364/1800 - Synchronizer modeling
- AMBA specifications - CDC requirements for multi-clock systems

---

Last Updated: 2026-04-21

6.10 Navigation

- Back to Shared Infrastructure Index
- Back to RTLAmba Index

7 Binary to Gray Code Converter

7.1 Overview

The `bin2gray` module implements a purely combinational binary-to-Gray code converter. Gray code (also known as reflected binary code or unit distance code) is a binary numeral system where two successive values differ in only one bit. This property makes Gray code essential for reducing glitches and metastability in digital systems, particularly in asynchronous designs and clock domain crossings.

7.2 Module Declaration

```
module bin2gray #(
    parameter int WIDTH = 4
) (
```

```
    input  wire [WIDTH-1:0] binary,
    output wire [WIDTH-1:0] gray
);
```

7.3 Parameters

7.3.1 WIDTH

- **Type**: `int`
- **Default**: `4`
- **Description**: Bit width of both input and output
- **Range**: Any positive integer ≥ 1
- **Common Values**: 4, 8, 16, 32 for address counters and data buses
- **Impact**: Determines the number of XOR gates required

7.4 Ports

7.4.1 Inputs

Port	Width	Type	Description
`binary`	WIDTH	`wire`	Input binary value

7.4.2 Outputs

Port	Width	Type	Description
`gray`	WIDTH	`wire`	Output Gray code value

7.5 Gray Code Theory

7.5.1 Mathematical Definition

Gray code conversion follows a simple mathematical relationship: - **MSB**: `gray[WIDTH-1] = binary[WIDTH-1]` (MSB unchanged) - **Other bits**: `gray[i] = binary[i] ⊕ binary[i+1]` for i = 0 to WIDTH-2

7.5.2 Why Gray Code?

Gray code solves several problems in digital systems:

1. **Single Bit Transitions**: Adjacent values differ by exactly one bit
2. **Glitch Reduction**: Eliminates intermediate states during transitions

3. **Metastability Prevention**: Safer for asynchronous clock domain crossings
4. **Mechanical Encoders**: Natural for optical/magnetic position encoders

7.5.3 Gray Code Sequence Examples

7.5.3.1 2-bit Gray Code

Decimal	Binary	Gray	Transition
0	00	00	-
1	01	01	bit 0
2	10	11	bit 1
3	11	10	bit 0

7.5.3.2 3-bit Gray Code

Decimal	Binary	Gray	Changed Bit
0	000	000	-
1	001	001	0
2	010	011	1
3	011	010	0
4	100	110	2
5	101	111	0
6	110	101	1
7	111	100	0

7.5.3.3 4-bit Gray Code (Complete Table)

Dec	Binary	Gray	Dec	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

7.6 Implementation Analysis

7.6.1 Core Logic

```
genvar i;
generate
    for (i = 0; i < WIDTH - 1; i++) begin : gen_gray
        assign gray[i] = binary[i] ^ binary[i+1];
    end
endgenerate

assign gray[WIDTH-1] = binary[WIDTH-1];
```

7.6.2 Logic Structure

The implementation creates a chain of XOR gates: - **Lower bits**: Each gray bit is XOR of current and next binary bits - **MSB**: Directly assigned from binary MSB - **Propagation**: Single level of logic (all XORs in parallel)

7.6.3 Gate-Level Implementation

For WIDTH = 4:

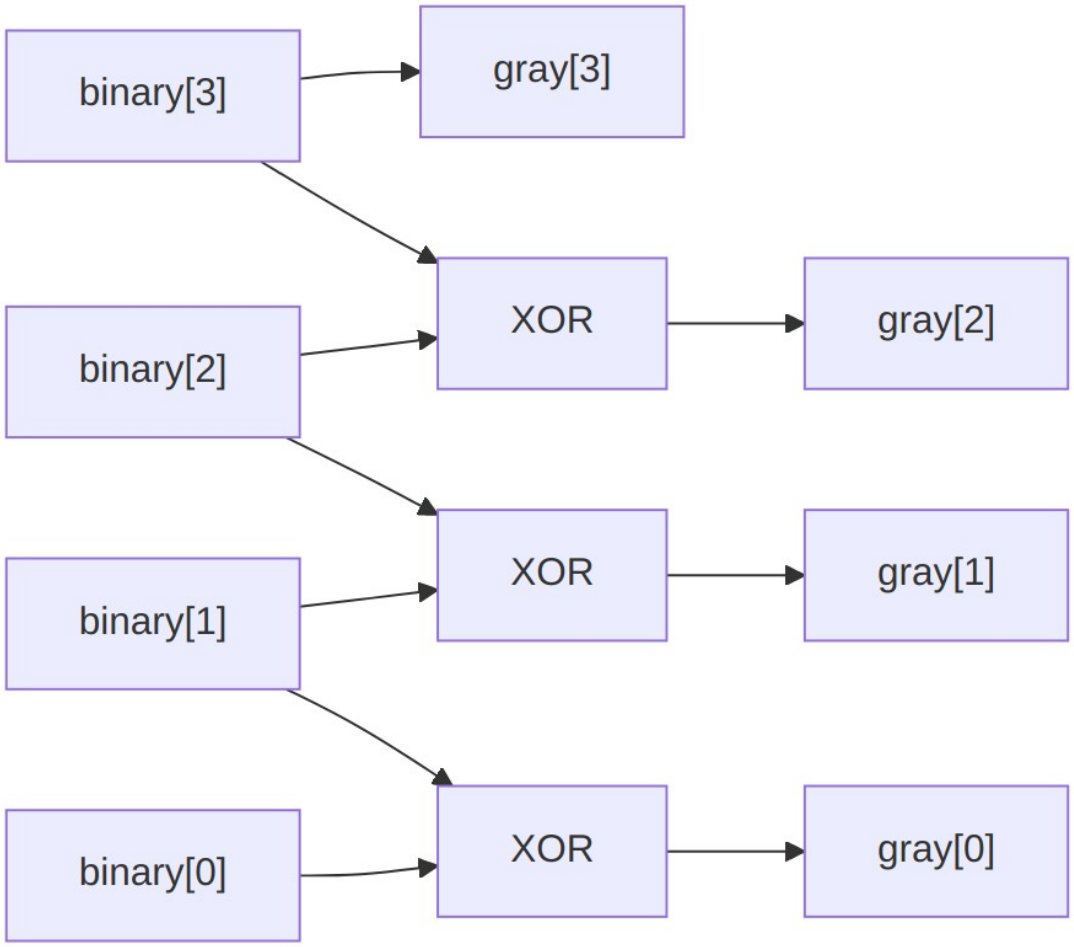


Diagram 2

7.6.4 Timing Characteristics

- **Propagation Delay**: Single XOR gate delay (typically 0.1-0.3ns in modern processes)
- **Critical Path**: Through one XOR gate only
- **Fan-out**: Each binary bit drives at most 2 XOR gates
- **Scalability**: Constant delay regardless of WIDTH

7.7 Design Examples and Applications

7.7.1 1. Asynchronous FIFO Pointer

```
module async_fifo_ptr #(
    parameter int ADDR_WIDTH = 4
) (
```

```
    input  logic                  clk,
    input  logic                  rst_n,
    input  logic                  enable,
    output logic [ADDR_WIDTH:0]   gray_ptr,
    output logic [ADDR_WIDTH-1:0] addr
);

    logic [ADDR_WIDTH:0] binary_ptr, binary_ptr_next;

    // Binary counter
    assign binary_ptr_next = enable ? (binary_ptr + 1) : binary_ptr;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            binary_ptr <= 'b0;
        else
            binary_ptr <= binary_ptr_next;
    end

    // Binary to Gray conversion
    bin2gray #(
        .WIDTH(ADDR_WIDTH + 1)
    ) ptr_converter (
        .binary(binary_ptr),
        .gray(gray_ptr)
    );

    // Extract address (lower bits of binary counter)
    assign addr = binary_ptr[ADDR_WIDTH-1:0];

endmodule
```

7.7.2 2. Clock Domain Crossing Counter

```
module cross_domain_counter #(
    parameter int WIDTH = 8
) (
    // Source domain
    input  logic             src_clk,
    input  logic             src_rst_n,
    input  logic             src_enable,

    // Destination domain
    input  logic             dst_clk,
    input  logic             dst_rst_n,
    output logic [WIDTH-1:0] dst_count_binary,
    output logic [WIDTH-1:0] dst_count_gray
);
```

```
    // Source domain counter
    logic [WIDTH-1:0] src_binary, src_gray;

    always_ff @(posedge src_clk or negedge src_rst_n) begin
        if (!src_rst_n)
            src_binary <= 'b0;
        else if (src_enable)
            src_binary <= src_binary + 1;
    end

    // Convert to Gray in source domain
    bin2gray #(.WIDTH(WIDTH)) src_converter (
        .binary(src_binary),
        .gray(src_gray)
    );

    // Synchronize Gray code to destination domain
    logic [WIDTH-1:0] dst_gray_sync;

    synchronizer #(.WIDTH(WIDTH)) gray_sync (
        .clk(dst_clk),
        .rst_n(dst_rst_n),
        .data_in(src_gray),
        .data_out(dst_gray_sync)
    );

    // Convert back to binary in destination domain
    gray2bin #(.WIDTH(WIDTH)) dst_converter (
        .gray(dst_gray_sync),
        .binary(dst_count_binary)
    );

    assign dst_count_gray = dst_gray_sync;

endmodule
```

7.7.3 3. Rotary Encoder Interface

```
module rotary_encoder_interface #(
    parameter int POSITION_WIDTH = 12
) (
    input  logic                          clk,
    input  logic                          rst_n,
    input  logic [POSITION_WIDTH-1:0]     encoder_binary,
    output logic [POSITION_WIDTH-1:0]     encoder_gray,
    output logic [POSITION_WIDTH-1:0]     position_filtered,
    output logic                          position_changed
```

```
);

    // Convert encoder binary position to Gray
    bin2gray #(
        .WIDTH(POSITION_WIDTH)
    ) encoder_conv (
        .binary(encoder_binary),
        .gray(encoder_gray)
    );

    // Synchronize and filter
    logic [POSITION_WIDTH-1:0] gray_sync, gray_prev;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            gray_sync <= 'b0;
            gray_prev <= 'b0;
        end else begin
            gray_sync <= encoder_gray;
            gray_prev <= gray_sync;
        end
    end

    // Detect changes (only one bit should change in Gray code)
    logic valid_transition;
    assign valid_transition = ($countones(gray_sync ^ gray_prev) <=
1);

    // Convert back to binary for position output
    gray2bin #(.WIDTH(POSITION_WIDTH)) pos_converter (
        .gray(gray_sync),
        .binary(position_filtered)
    );

    assign position_changed = valid_transition && (gray_sync !=
gray_prev);

endmodule
```

7.7.4 4. Address Scrambling for Memory Testing

```
module memory_address_scrambler #(
    parameter int ADDR_WIDTH = 16
) (
    input  logic [ADDR_WIDTH-1:0] linear_addr,
    output logic [ADDR_WIDTH-1:0] scrambled_addr
);
```

```
    logic [ADDR_WIDTH-1:0] gray_addr;

    // Convert linear address to Gray code for scrambling
    bin2gray #(.WIDTH(ADDR_WIDTH)) scrambler (
        .binary(linear_addr),
        .gray(gray_addr)
    );

    // Additional scrambling (optional)
    assign scrambled_addr = {gray_addr[0], gray_addr[ADDR_WIDTH-1:1]};

endmodule
```

7.7.5 5. Multi-Bit Synchronizer with Gray Code

```
module gray_code_synchronizer #(
    parameter int WIDTH = 8,
    parameter int SYNC_STAGES = 2
) (
    input  logic             src_clk,
    input  logic             src_rst_n,
    input  logic [WIDTH-1:0] src_data,

    input  logic             dst_clk,
    input  logic             dst_rst_n,
    output logic [WIDTH-1:0] dst_data
);

    // Convert to Gray in source domain
    logic [WIDTH-1:0] src_gray;

    bin2gray #(.WIDTH(WIDTH)) src_conv (
        .binary(src_data),
        .gray(src_gray)
    );

    // Multi-stage synchronizer for Gray code
    logic [WIDTH-1:0] sync_regs [SYNC_STAGES];

    always_ff @(posedge dst_clk or negedge dst_rst_n) begin
        if (!dst_rst_n) begin
            for (int i = 0; i < SYNC_STAGES; i++) begin
                sync_regs[i] <= 'b0;
            end
        end else begin
            sync_regs[0] <= src_gray;
            for (int i = 1; i < SYNC_STAGES; i++) begin
                sync_regs[i] <= sync_regs[i-1];
```

```
            end
        end
    end

    // Convert back to binary in destination domain
    gray2bin #(.WIDTH(WIDTH)) dst_conv (
        .gray(sync_regs[SYNC_STAGES-1]),
        .binary(dst_data)
    );

endmodule
```

7.8 Companion Gray-to-Binary Converter

7.8.1 Implementation

```
module gray2bin #(
    parameter int WIDTH = 4
) (
    input  wire [WIDTH-1:0] gray,
    output wire [WIDTH-1:0] binary
);

    genvar i;

    // MSB is unchanged
    assign binary[WIDTH-1] = gray[WIDTH-1];

    // Other bits: XOR accumulation from MSB down
    generate
        for (i = WIDTH-2; i >= 0; i--) begin : gen_binary
            assign binary[i] = binary[i+1] ^ gray[i];
        end
    endgenerate

endmodule
```

7.8.2 Gray-to-Binary Truth Table (4-bit)

Gray	Binary	Conversion Process
0000	0000	bin[3]=0, bin[2]=0⊕0=0, bin[1]=0⊕0=0, bin[0]=0⊕0=0
0001	0001	bin[3]=0, bin[2]=0⊕0=0,

Gray	Binary	Conversion Process
		bin[1]=0⊕0=0, bin[0]=0⊕1=1
0011	0010	bin[3]=0, bin[2]=0⊕0=0, bin[1]=0⊕1=1, bin[0]=1⊕1=0
0010	0011	bin[3]=0, bin[2]=0⊕0=0, bin[1]=0⊕1=1, bin[0]=1⊕0=1

7.9 Advanced Implementations

7.9.1 1. Parameterized Converter with Validation

```
module bin2gray_validated #(
    parameter int WIDTH = 4,
    parameter bit ENABLE_CHECKS = 1
) (
    input  logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray,
    output logic             valid
);

    // Basic conversion
    bin2gray #(.WIDTH(WIDTH)) converter (
        .binary(binary),
        .gray(gray)
    );

    // Optional validation
    generate
        if (ENABLE_CHECKS) begin : validation
            logic [WIDTH-1:0] binary_check;

            // Round-trip conversion check
            gray2bin #(.WIDTH(WIDTH)) check_converter (
                .gray(gray),
                .binary(binary_check)
            );

            assign valid = (binary == binary_check);
        end else begin : no_validation
```

```
            assign valid = 1'b1;
        end
    endgenerate

endmodule
```

7.9.2 2. Pipelined Converter for High Speed

```
module bin2gray_pipelined #(
    parameter int WIDTH = 32,
    parameter int PIPELINE_STAGES = 2
) (
    input  logic             clk,
    input  logic             rst_n,
    input  logic [WIDTH-1:0] binary_in,
    input  logic             valid_in,
    output logic [WIDTH-1:0] gray_out,
    output logic             valid_out
);

    // Pipeline registers
    logic [WIDTH-1:0] binary_pipe [PIPELINE_STAGES];
    logic valid_pipe [PIPELINE_STAGES];
    logic [WIDTH-1:0] gray_pipe [PIPELINE_STAGES];

    // Stage 0: Input registration
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            binary_pipe[0] <= 'b0;
            valid_pipe[0] <= 1'b0;
        end else begin
            binary_pipe[0] <= binary_in;
            valid_pipe[0] <= valid_in;
        end
    end

    // Conversion in stage 0
    bin2gray #(.WIDTH(WIDTH)) stage0_conv (
        .binary(binary_pipe[0]),
        .gray(gray_pipe[0])
    );

    // Additional pipeline stages
    generate
        for (genvar s = 1; s < PIPELINE_STAGES; s++) begin :
pipeline_stages
            always_ff @(posedge clk or negedge rst_n) begin
                if (!rst_n) begin
```

```
                    gray_pipe[s] <= 'b0;
                    valid_pipe[s] <= 1'b0;
                end else begin
                    gray_pipe[s] <= gray_pipe[s-1];
                    valid_pipe[s] <= valid_pipe[s-1];
                end
            end
        end
    endgenerate

    // Output assignment
    assign gray_out = gray_pipe[PIPELINE_STAGES-1];
    assign valid_out = valid_pipe[PIPELINE_STAGES-1];

endmodule
```

7.9.3 3. Bidirectional Converter

```
module bidirectional_gray_converter #(
    parameter int WIDTH = 8
) (
    input  logic             direction,  // 0: bin→gray, 1: gray→bin
    input  logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);

    logic [WIDTH-1:0] bin_to_gray_out, gray_to_bin_out;

    // Both converters always active
    bin2gray #(.WIDTH(WIDTH)) b2g (
        .binary(data_in),
        .gray(bin_to_gray_out)
    );

    gray2bin #(.WIDTH(WIDTH)) g2b (
        .gray(data_in),
        .binary(gray_to_bin_out)
    );

    // Mux output based on direction
    assign data_out = direction ? gray_to_bin_out : bin_to_gray_out;

endmodule
```

7.10 Verification and Testing

7.10.1 Comprehensive Test Bench

```
module tb_bin2gray;

    parameter int WIDTH = 4;
    parameter int MAX_VAL = (1 << WIDTH) - 1;

    logic [WIDTH-1:0] binary, gray;
    logic [WIDTH-1:0] expected_gray;
    logic [WIDTH-1:0] binary_check;

    // DUT
    bin2gray #(.WIDTH(WIDTH)) dut (
        .binary(binary),
        .gray(gray)
    );

    // Reference converter for checking
    gray2bin #(.WIDTH(WIDTH)) check_conv (
        .gray(gray),
        .binary(binary_check)
    );

    // Test sequence
    initial begin
        $display("Testing %d-bit Binary to Gray converter", WIDTH);

        // Test all possible values
        for (int i = 0; i <= MAX_VAL; i++) begin
            binary = i;
            expected_gray = compute_gray_reference(i);

            #1; // Allow propagation

            // Check conversion correctness
            if (gray !== expected_gray) begin
                $error("Mismatch: binary=%b, expected_gray=%b,
actual_gray=%b",
                       binary, expected_gray, gray);
            end

            // Check round-trip conversion
            if (binary_check !== binary) begin
                $error("Round-trip failed: original=%b, recovered=%b",
                       binary, binary_check);
            end
```

```
            // Check single-bit change property
            if (i > 0) begin
                logic [WIDTH-1:0] prev_gray =
compute_gray_reference(i-1);
                int bit_changes = $countones(gray ^ prev_gray);
                if (bit_changes != 1) begin
                    $error("Multiple bit change: %d→%d, gray %b→%b,
changes=%d",
                           i-1, i, prev_gray, gray, bit_changes);
                end
            end

            $display("PASS: %d → binary:%b → gray:%b", i, binary,
gray);
        end

        $display("All tests passed!");
        $finish;
    end

    // Reference Gray code computation
    function [WIDTH-1:0] compute_gray_reference(input [WIDTH-1:0]
bin);
        compute_gray_reference[WIDTH-1] = bin[WIDTH-1];
        for (int i = WIDTH-2; i >= 0; i--) begin
            compute_gray_reference[i] = bin[i] ^ bin[i+1];
        end
    endfunction

endmodule
```

7.10.2 Property-Based Verification

```
module bin2gray_properties #(
    parameter int WIDTH = 4
);

    logic [WIDTH-1:0] binary, gray;

    // Bind to DUT
    bind bin2gray bin2gray_properties #(WIDTH) props_inst (.*);

    // Property: MSB preservation
    property msb_unchanged;
        gray[WIDTH-1] == binary[WIDTH-1];
    endproperty
```

```
    // Property: Single bit change between consecutive values
    property single_bit_change;
        logic [WIDTH-1:0] bin_plus_one = binary + 1;
        logic [WIDTH-1:0] gray_plus_one;

        // Compute Gray code for binary+1
        assign gray_plus_one[WIDTH-1] = bin_plus_one[WIDTH-1];
        for (genvar i = 0; i < WIDTH-1; i++) begin
            assign gray_plus_one[i] = bin_plus_one[i] ^
bin_plus_one[i+1];
        end

        // Check single bit difference
        (binary < (2**WIDTH - 1)) |-> ($countones(gray ^
gray_plus_one) == 1);
    endproperty

    // Property: Round-trip conversion
    property round_trip_conversion;
        logic [WIDTH-1:0] recovered_binary;

        // Convert back to binary
        assign recovered_binary[WIDTH-1] = gray[WIDTH-1];
        for (genvar i = WIDTH-2; i >= 0; i--) begin
            assign recovered_binary[i] = recovered_binary[i+1] ^
gray[i];
        end

        recovered_binary == binary;
    endproperty

    // Assertions
    assert property (msb_unchanged);
    assert property (single_bit_change);
    assert property (round_trip_conversion);

endmodule
```

7.10.3 Coverage Model

```
covergroup bin2gray_cg;

    cp_binary_values: coverpoint binary {
        bins zero = {0};
        bins powers_of_two[] = {1, 2, 4, 8, 16}; // For appropriate
WIDTH
        bins max_value = {2**WIDTH - 1};
        bins mid_range[] = {[1:2**(WIDTH-1)-1]};
        bins upper_range[] = {[2**(WIDTH-1):2**WIDTH-2]};
```

```
    }

    cp_gray_values: coverpoint gray {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_bit_patterns: coverpoint binary {
        bins alternating_01 = {8'b01010101}; // For WIDTH=8
        bins alternating_10 = {8'b10101010};
        bins all_ones = {'1};
        bins all_zeros = {'0};
    }

    // Cross coverage between input patterns and outputs
    cross_binary_gray: cross cp_binary_values, cp_gray_values;

endcovergroup
```

7.11 Synthesis Optimization

7.11.1 Resource Utilization

For different widths, typical FPGA resource usage:

WIDTH	LUTs	Delay (ns)	Max Freq (MHz)
4	3	0.2	800+
8	7	0.2	800+
16	15	0.3	600+
32	31	0.4	500+

7.11.2 Timing Optimization

```
// For critical timing paths, add pipeline register
module bin2gray_registered #(
    parameter int WIDTH = 16
) (
    input  logic             clk,
    input  logic             rst_n,
    input  logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray
);

    logic [WIDTH-1:0] gray_comb;

    // Combinational conversion
    bin2gray #(.WIDTH(WIDTH)) conv (
```

```
        .binary(binary),
        .gray(gray_comb)
    );

    // Output register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            gray <= 'b0;
        else
            gray <= gray_comb;
    end

endmodule
```

7.11.3 Power Optimization

```
// Clock gating for power savings
module bin2gray_gated #(
    parameter int WIDTH = 8
) (
    input  logic             clk,
    input  logic             rst_n,
    input  logic             enable,
    input  logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray
);

    logic gated_clk;

    // Clock gate
    clock_gate cg_inst (
        .clk(clk),
        .enable(enable),
        .gated_clk(gated_clk)
    );

    // Registered converter
    bin2gray_registered #(.WIDTH(WIDTH)) conv (
        .clk(gated_clk),
        .rst_n(rst_n),
        .binary(binary),
        .gray(gray)
    );

endmodule
```

7.12 Common Applications Summary

7.12.1 1. Asynchronous FIFOs: Prevent metastability in pointer comparisons

7.12.2 2. Clock Domain Crossing: Safe multi-bit signal transfer

7.12.3 3. Position Encoders: Mechanical/optical encoder interfaces

7.12.4 4. Memory Address Generation: Reduce EMI and power spikes

7.12.5 5. Test Pattern Generation: Controlled single-bit transitions

7.12.6 6. Error Detection: Single-bit error detection schemes

7.12.7 7. ADC/DAC Interfaces: Reduce glitches in conversion systems

7.13 Design Guidelines

7.13.1 1. Always Use for Async Boundaries: Gray code is essential for safe asynchronous transfers

7.13.2 2. Consider Timing: Purely combinational - no setup/hold concerns

7.13.3 3. Verify Single-Bit Property: Always verify adjacent values differ by one bit

7.13.4 4. Plan for Back-Conversion: Usually need Gray-to-Binary converter too

7.13.5 5. Check Width Requirements: Ensure sufficient bits for application range

The Binary-to-Gray converter is a fundamental building block in digital design, providing robust solutions for asynchronous interfaces and glitch-free operations. Its simplicity and reliability make it indispensable for safe digital system design.

7.14 Navigation

- ← Back to RTLCommon Index
- ← Back to Main Documentation Index

8 Gray-to-Binary Converter (gray2bin.sv)

8.1 Purpose

Converts Gray code (reflected binary code) to standard binary representation using XOR reduction. Essential component for asynchronous FIFO pointer comparison and other CDC applications.

8.2 Ports

8.2.1 Input Ports

- **gray[WIDTH-1:0]** - Gray code input

8.2.2 Output Ports

- **binary[WIDTH-1:0]** - Binary code output

8.2.3 Parameters

- **WIDTH** - Data width in bits (default: 4)

8.3 Gray Code Theory

8.3.1 What is Gray Code?

Gray code (also called reflected binary code) is a binary numeral system where **only one bit changes** between consecutive values:

```
Binary  Gray   Transitions
000  →  000
001  →  001    1 bit change
010  →  011    1 bit change
011  →  010    1 bit change
100  →  110    1 bit change
101  →  111    1 bit change
110  →  101    1 bit change
111  →  100    1 bit change
```

8.3.2 Why Gray Codes Matter for CDC

- **Metastability protection**: Only one bit transition eliminates multi-bit race conditions
- **Safe sampling**: Intermediate values during transition are still valid Gray codes
- **FIFO pointers**: Essential for async FIFO full/empty detection

8.4 Conversion Algorithm

8.4.1 Mathematical Foundation

For Gray-to-binary conversion, each binary bit is the XOR of all Gray bits from that position to the MSB:

```
binary[i] = gray[MSB] ⊕ gray[MSB-1] ⊕ ... ⊕ gray[i]
```

8.4.2 Implementation

```
genvar i;
generate
    for (i = 0; i < WIDTH; i++) begin : gen_gray_to_bin
        assign binary[i] = ^(gray >> i);
    end
endgenerate
```

8.4.3 Bit-by-Bit Breakdown

```
// For 4-bit example:
assign binary[3] = gray[3];                          // MSB unchanged
assign binary[2] = gray[3] ^ gray[2];                // XOR from MSB
down
assign binary[1] = gray[3] ^ gray[2] ^ gray[1];      // XOR from MSB
down
assign binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0]; // XOR all
bits
```

8.5 Functional Examples

8.5.1 4-Bit Conversion Table

Gray[3:0]	Binary[3:0]	Calculation
0000	0000	0⊕0⊕0⊕0=0, 0⊕0⊕0=0, 0⊕0=0, 0=0
0001	0001	0⊕0⊕0⊕1=1, 0⊕0⊕0=0, 0⊕0=0, 0=0
0011	0010	0⊕0⊕1⊕1=0, 0⊕0⊕1=1, 0⊕0=0, 0=0
0010	0011	0⊕0⊕1⊕0=1, 0⊕0⊕1=1, 0⊕0=0,

Gray[3:0]	Binary[3:0]	Calculation
		0=0
0110	0100	0⊕1⊕1⊕0=0, 0⊕1⊕1=0, 0⊕1=1, 0=0
0111	0101	0⊕1⊕1⊕1=1, 0⊕1⊕1=0, 0⊕1=1, 0=0
0101	0110	0⊕1⊕0⊕1=0, 0⊕1⊕0=1, 0⊕1=1, 0=0
0100	0111	0⊕1⊕0⊕0=1, 0⊕1⊕0=1, 0⊕1=1, 0=0

8.5.2 Step-by-Step Example (Gray 0110 → Binary)

```
Gray input: 0110

binary[3] = gray[3] = 0
binary[2] = gray[3] ^ gray[2] = 0 ^ 1 = 1
binary[1] = gray[3] ^ gray[2] ^ gray[1] = 0 ^ 1 ^ 1 = 0
binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0] = 0 ^ 1 ^ 1 ^ 0 = 0

Result: binary = 0100 (decimal 4)
```

8.6 Implementation Analysis

8.6.1 XOR Reduction Technique

```
assign binary[i] = ^(gray >> i);
```

How it works: - `gray >> i`: Right-shift gray by i positions - `^(...)`: XOR-reduce all bits in the shifted value - **Effect**: XORs all bits from position i to MSB

8.6.2 Generate Loop Benefits

- **Parameterizable**: Works for any WIDTH
- **Synthesis friendly**: Tools recognize and optimize pattern
- **Parallel computation**: All bits calculated simultaneously
- **Resource efficient**: Maps to XOR tree structures

8.7 Use Cases in Digital Design

8.7.1 Asynchronous FIFO Pointers

```
// Convert synchronized Gray pointers back to binary for comparison
gray2bin #(.WIDTH(5)) wr_ptr_conv (
    .gray(sync_wr_ptr_gray),
    .binary(sync_wr_ptr_bin)
);

gray2bin #(.WIDTH(5)) rd_ptr_conv (
    .gray(sync_rd_ptr_gray),
    .binary(sync_rd_ptr_bin)
);

// Now can perform binary arithmetic for occupancy calculation
assign occupancy = sync_wr_ptr_bin - sync_rd_ptr_bin;
```

8.7.2 Clock Domain Crossing Counters

```
// Convert Gray counter back to binary for address generation
gray2bin #(.WIDTH(AW)) addr_converter (
    .gray(gray_counter),
    .binary(memory_address)
);
```

8.7.3 Position Encoding

```
// Convert Gray-encoded position to binary for processing
gray2bin #(.WIDTH(8)) position_decoder (
    .gray(gray_position),
    .binary(bin_position)
);
```

8.8 Timing Characteristics

8.8.1 Propagation Delay

- **XOR tree depth**: `log2(WIDTH)` levels
- **Typical delay**: 1-2 LUT delays for most widths
- **Critical path**: From MSB input to LSB output

8.8.2 Delay Analysis by Width

WIDTH	XOR Levels	Typical Delay
4	2	1 LUT delay
8	3	2 LUT delays

WIDTH	XOR Levels	Typical Delay
16	4	2 LUT delays
32	5	3 LUT delays

8.8.3 Synthesis Optimization

Modern synthesis tools: - **Recognize pattern**: Optimize XOR reduction automatically - **Balance trees**: Create balanced XOR structures - **Resource sharing**: Reuse XOR gates where possible

8.9 Design Considerations

8.9.1 Width Scaling

- **Linear resource growth**: XOR gates increase with WIDTH
- **Logarithmic delay growth**: Delay scales with log(WIDTH)
- **Practical limits**: Works well up to 64+ bits

8.9.2 Input Validation

```
// Gray codes have no invalid states - all inputs are valid
// However, may want to validate source of Gray code
always_comb begin
    if (gray_code_valid) begin
        binary_out = converted_binary;
    end else begin
        binary_out = '0;  // Default when invalid
    end
end
```

8.9.3 Pipeline Considerations

For very high-speed applications:

```
// Optional pipeline stage for timing closure
always_ff @(posedge clk) begin
    if (!rst_n) begin
        binary_reg <= '0;
    end else begin
        binary_reg <= binary_comb;
    end
end
```

8.10 Verification Strategies

8.10.1 Exhaustive Testing

```
// For reasonable widths, test all possible inputs
for (int i = 0; i < (1 << WIDTH); i++) begin
    gray_input = int_to_gray(i);      // Convert integer to Gray
    expected_binary = i;              // Expected result
    #1;
    assert(binary_output == expected_binary);
end
```

8.10.2 Property-Based Verification

```
// Verify round-trip conversion
property round_trip;
    @(posedge clk)
    binary_output == original_binary;
endproperty

// Apply after binary→Gray→binary conversion
assert property (round_trip);
```

8.10.3 Random Testing

```
repeat (10000) begin
    random_gray = $random();
    expected_result = reference_gray2bin(random_gray);
    #1;
    assert(binary_output == expected_result);
end
```

8.11 Common Mistakes and Pitfalls

8.11.1 Bit Order Confusion

```
// WRONG: Bit order matters in Gray codes
assign binary[i] = ^(gray << i);  // Left shift instead of right

// CORRECT:
assign binary[i] = ^(gray >> i);  // Right shift
```

8.11.2 Width Mismatches

```
// WRONG: Input/output width mismatch
gray2bin #(.WIDTH(4)) converter (
    .gray(gray_5bit),     // 5-bit input
    .binary(binary_4bit)  // 4-bit output
);
```

```
// CORRECT: Match widths
gray2bin #(.WIDTH(5)) converter (
    .gray(gray_5bit),
    .binary(binary_5bit)
);
```

8.11.3 Timing Assumptions

```
// WRONG: Assuming zero delay
gray_in <= new_value;
binary_out <= converted_value;  // May not be updated yet

// CORRECT: Account for combinational delay
gray_in <= new_value;
#1;  // Wait for conversion
binary_out <= converted_value;
```

8.12 Related Modules

- **bin2gray**: Performs inverse conversion (binary to Gray)
- **counter_bingray**: Combined binary/Gray counter
- **fifo_async**: Uses Gray codes for CDC
- **grayj2bin**: Johnson counter to binary converter (different algorithm)

8.13 Navigation

- **← Back to RTLCommon Index**
- **← Back to Main Documentation Index**

9 Johnson-to-Binary Converter (`grayj2bin.sv`)

9.1 Purpose

Converts Johnson counter codes to binary representation for use in asynchronous FIFOs with non-power-of-2 depths. Unlike standard Gray-to-binary conversion, this module handles the unique properties of Johnson counter sequences.

9.2 Ports

9.2.1 Input Ports

- **`clk`** - Clock signal (required for leading/trailing one detection)
- **`rst_n`** - Active-low reset

- **`gray[JCW-1:0]`** - Johnson counter input (misleadingly named "gray")

9.2.2 Output Ports

- **`binary[WIDTH-1:0]`** - Binary output representing position in sequence

9.2.3 Parameters

- **`JCW`** - Johnson Counter Width (equals FIFO DEPTH)
- **`WIDTH`** - Binary output width (typically `$clog2(DEPTH) + 1`)
- **`INSTANCE_NAME`** - String identifier for debug

9.3 Johnson Counter Sequence Review

9.3.1 Johnson Counter Characteristics

Johnson counters are **shift registers with inverted feedback**:

```
For JCW=6 (DEPTH=6):
State 0:  000000  ← Initial state
State 1:  100000  ← Shift left, insert 1
State 2:  110000
State 3:  111000
State 4:  111100
State 5:  111110
State 6:  111111  ← All 1s (special case)
State 7:  011111  ← Shift left, insert ~MSB (0)
State 8:  001111
State 9:  000111
State 10: 000011
State 11: 000001
State 0:  000000  ← Cycle complete (12 states total)
```

9.3.2 Key Properties

- **Single bit transitions**: Only one bit changes per state
- **Two phases**:
  - **First half** (0 to DEPTH-1): Filling with 1s from left
  - **Second half** (DEPTH to 2×DEPTH-1): Emptying 1s from left
- **Wrap indicator**: MSB indicates which half of cycle

9.4 Conversion Algorithm

9.4.1 Strategy Overview

The conversion uses **position detection** of the transition between 1s and 0s:

```
if (w_all_zeroes || w_all_ones) begin
    w_binary = {WIDTH{1'b0}};
end else if (gray[JCW-1]) begin
    // Second half: use leading one position directly
    w_binary = {{(WIDTH-N){1'b0}}, w_trailing_one};
end else begin
    // First half: use trailing one + 1
    w_binary = {{(WIDTH-N){1'b0}}, (w_leading_one + 1'b1)};
end
```

9.4.2 Position Detection Module

```
leading_one_trailing_one #(
    .WIDTH(JCW),
    .INSTANCE_NAME(INSTANCE_NAME)
) u_leading_one_trailing_one (
    .data(gray),
    .leadingone(w_leading_one),
    .trailingone(w_trailing_one),
    .all_zeroes(w_all_zeroes),
    .all_ones(w_all_ones),
    .valid(w_valid)
);
```

9.5 Detailed Conversion Examples

9.5.1 First Half Conversion (MSB = 0)

```
Johnson: 001111 (w_leading_one = 5, w_trailing_one = 2)
Logic: First half, so position = w_leading_one + 1 = 5 + 1 = 6
Binary: 000110 (with MSB=0 indicating first half)
```

9.5.2 Second Half Conversion (MSB = 1)

```
Johnson: 111000 (w_leading_one = 0, w_trailing_one = 2)
Logic: Second half, so position = w_trailing_one = 2
Binary: 100010 (with MSB=1 indicating second half)
```

9.5.3 Special Cases

```
Johnson: 000000 → Binary: 000000 (all zeros case)
Johnson: 111111 → Binary: 000000 (all ones case - same as zeros)
```

9.6 Implementation Deep Dive

9.6.1 Three-Part Binary Construction

```
assign binary[WIDTH-1]   = gray[JCW-1];                    // MSB = wrap
indicator
```

```
assign binary[WIDTH-2:0] = w_binary[WIDTH-2:0];          // Lower bits
= position
```

9.6.2 Width Calculations

```
localparam int N = $clog2(JCW);                          // Address
bits needed
localparam int PAD_WIDTH = (WIDTH > N+1) ? WIDTH-N-1 : 0; // Padding
if needed
```

9.6.3 Why This Algorithm Works

9.6.3.1 First Half Logic (MSB = 0)

- **Pattern**: `000...0111...1` (0s followed by 1s)
- **Leading one**: Position of leftmost 1
- **Conversion**: Position = leading_one + 1
- **Reasoning**: We've filled (leading_one + 1) positions with 1s

9.6.3.2 Second Half Logic (MSB = 1)

- **Pattern**: `111...1000...0` (1s followed by 0s)

- **Trailing one**: Position of rightmost 1
- **Conversion**: Position = trailing_one
- **Reasoning**: We're emptying from the left, trailing_one shows how far

9.7 Use in Asynchronous FIFO

9.7.1 Context in FIFO Operation

```
// fifo_async_div2.sv usage:
grayj2bin #(
    .JCW(JCW),                    // = DEPTH
    .WIDTH(AW + 1),               // Address width + wrap bit
    .INSTANCE_NAME("rd_ptr_gray2bin_inst")
) rd_ptr_gray2bin_inst(
    .binary(w_wdom_rd_ptr_bin),   // Binary for arithmetic
    .gray(r_wdom_rd_ptr_gray),    // Johnson counter from CDC
    .clk(wr_clk),
    .rst_n(wr_rst_n)
);
```

9.7.2 Why Clock is Required

Unlike pure combinational Gray-to-binary conversion, Johnson-to-binary needs the `leading_one_trailing_one` helper, which may use sequential logic for complex position detection.

9.8 Comparison with Standard Gray2Bin

Aspect	Standard Gray2Bin	Johnson2Bin (grayj2bin)
Input type	Traditional Gray code	Johnson counter sequence
Algorithm	XOR reduction	Position detection
Complexity	Simple combinational	Complex position logic
Clock requirement	None	Required for helper modules
Width scaling	Logarithmic	Linear with JCW
Use case	Power-of-2 sequences	Any even sequences

9.9 Performance Characteristics

9.9.1 Timing Analysis

- **Critical path**: Through position detection logic
- **Delay components**:
  - Leading/trailing one detection
  - Binary arithmetic (addition)
  - Output multiplexing
- **Synthesis complexity**: Higher than standard Gray conversion

9.9.2 Resource Utilization

```
Resources scale with JCW (Johnson Counter Width):
- Small FIFOs (JCW ≤ 16): Reasonable overhead
- Medium FIFOs (JCW = 32-64): Significant resources
- Large FIFOs (JCW > 64): May become limiting factor
```

9.10 Design Considerations

9.10.1 When to Use Johnson vs. Gray

```
// Use Johnson counter approach when:
parameter int DEPTH = 10;  // Non-power-of-2 required
parameter int DEPTH = 6;   // Specific size needed
parameter int DEPTH = 14;  // Standard Gray won't work

// Use standard Gray approach when:
parameter int DEPTH = 8;   // Power-of-2 is acceptable
parameter int DEPTH = 16;  // Efficiency more important
parameter int DEPTH = 64;  // Large depth, resource conscious
```

9.10.2 Resource Planning

```
// Resource overhead estimation:
// Standard Gray: ~log2(DEPTH) XOR gates
// Johnson: ~DEPTH comparators + position logic

// Break-even point typically around DEPTH = 16-32
```

9.10.3 Verification Challenges

```
// More complex verification due to:
// 1. Two-phase sequence behavior
// 2. Position detection correctness
// 3. Special case handling (all 0s, all 1s)
// 4. Wraparound boundary conditions
```

9.11 Error Conditions and Debug

9.11.1 Invalid Johnson Sequences

The `leading_one_trailing_one` module provides validation:

```
.valid(w_valid)  // Indicates valid Johnson pattern
```

Valid Johnson patterns have exactly one transition from 1s to 0s (or vice versa).

9.11.2 Debug Visibility

```
// Monitor internal state for debugging:
// - w_leading_one: Position of leftmost 1
// - w_trailing_one: Position of rightmost 1
// - w_all_zeroes: Special case flag
// - w_all_ones: Special case flag
// - w_valid: Sequence validity
```

9.11.3 Common Issues

1. **Invalid sequences**: Non-Johnson patterns on input
2. **Width mismatches**: JCW vs. actual Johnson counter width
3. **Phase confusion**: Misunderstanding first vs. second half logic
4. **Boundary conditions**: All-zeros and all-ones handling

9.12 Related Modules

- **counter_johnson**: Generates Johnson counter sequences
- **leading_one_trailing_one**: Position detection helper
- **fifo_async_div2**: Primary user of this conversion
- **gray2bin**: Standard Gray-to-binary for comparison
- **fifo_control**: Uses converted binary values for status generation

9.13 Advanced Topics

9.13.1 Hierarchical Johnson Counters

For very large depths, consider hierarchical approach:

```
// Break large Johnson counter into smaller segments
// Use multiple grayj2bin instances with higher-level arbitration
```

9.13.2 Alternative Position Detection

Some implementations use different position detection algorithms: - **Priority encoders**: Find first/last set bit - **Thermometer decoders**: Convert 1-hot positions - **LUT-based**: For small, fixed widths

9.14 Navigation

- **← Back to RTLCommon Index**
- **← Back to Main Documentation Index**

10 Binary-Gray Counter Module

10.1 Overview

The `counter_bingray` module is a dual-output counter that simultaneously provides both binary and Gray code representations of the same count value. This is essential for asynchronous FIFO implementations where Gray code is required to prevent metastability when crossing clock domains, while binary values are needed for arithmetic operations.

10.2 Module Declaration

```
module counter_bingray #(
    parameter int WIDTH = 4
) (
    input  logic             clk,
    input  logic             rst_n,
    input  logic             enable,
    output logic [WIDTH-1:0] counter_bin,
    output logic [WIDTH-1:0] counter_bin_next,
    output logic [WIDTH-1:0] counter_gray
);
```

10.3 Parameters

10.3.1 WIDTH

- **Type**: `int`
- **Default**: `4`
- **Description**: Bit width of both binary and Gray code outputs
- **Range**: Any positive integer ≥ 1
- **Impact**: Determines maximum count value (2^WIDTH - 1)

10.4 Ports

10.4.1 Inputs

Port	Width	Type	Description
`clk`	1	`logic`	System clock input
`rst_n`	1	`logic`	Active-low asynchronous reset
`enable`	1	`logic`	Counter enable control

10.4.2 Outputs

Port	Width	Type	Description
`counter_bin`	WIDTH	`logic`	Binary counter output (registered)
`counter_bin`	WIDTH	`logic`	Next binary value

Port	Width	Type	Description
_next			(combinational)
counter_gray	WIDTH	logic	Gray code output (registered)

10.5 Architecture Details

10.5.1 Internal Logic

```
logic [WIDTH-1:0] w_counter_bin, w_counter_gray;

assign w_counter_bin = enable ? (counter_bin + 1) : counter_bin;
assign w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
assign counter_bin_next = w_counter_bin;
```

10.5.2 Gray Code Conversion

The module implements the standard binary-to-Gray conversion: - **MSB**: `gray[WIDTH-1] = binary[WIDTH-1]` - **Other bits**: `gray[i] = binary[i] ^ binary[i+1]` for i = 0 to WIDTH-2

10.6 Gray Code Theory

10.6.1 Why Gray Code?

Gray code (reflected binary code) has the property that adjacent values differ by exactly one bit. This is crucial for:

1. **Metastability Prevention**: When crossing clock domains, only one bit changes at a time
2. **Glitch Elimination**: Reduces intermediate states during transitions
3. **Asynchronous Safety**: Safe for use in asynchronous circuits

10.6.2 Gray Code Sequence Example (4-bit)

Decimal	Binary	Gray	Changes
0	0000	0000	-
1	0001	0001	bit 0
2	0010	0011	bit 1
3	0011	0010	bit 0
4	0100	0110	bit 2
5	0101	0111	bit 0
6	0110	0101	bit 1

Decimal	Binary	Gray	Changes
7	0111	0100	bit 0
8	1000	1100	bit 3
9	1001	1101	bit 0
10	1010	1111	bit 1
11	1011	1110	bit 0
12	1100	1010	bit 2
13	1101	1011	bit 0
14	1110	1001	bit 1
15	1111	1000	bit 0

10.7 Implementation Analysis

10.7.1 Combinational Logic

```
// Next binary calculation
w_counter_bin = enable ? (counter_bin + 1) : counter_bin;

// Gray code conversion
w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
```

10.7.2 Sequential Logic

```
always_ff @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
        counter_bin  <= 'b0;
        counter_gray <= 'b0;
    end else begin
        counter_bin  <= w_counter_bin;
        counter_gray <= w_counter_gray;
    end
end
```

10.8 Timing Characteristics

10.8.1 Critical Paths

1. **Binary Increment**: Standard binary addition timing
2. **Gray Conversion**: XOR tree with log(WIDTH) levels
3. **Combined Path**: Increment + conversion in same cycle

10.8.2 Propagation Delays

- **Clock-to-Q**: Standard flip-flop delay
- **Combinational**: Depends on addition and XOR logic depth
- **Setup Time**: Must account for longest combinational path

10.9 Application: Asynchronous FIFO

10.9.1 FIFO Pointer Implementation

```
// Write domain counter
counter_bingray #(.WIDTH(ADDR_WIDTH+1)) wr_counter (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(wr_enable),
    .counter_bin(wr_bin),
    .counter_bin_next(wr_bin_next),
    .counter_gray(wr_gray)
);

// Read domain counter
counter_bingray #(.WIDTH(ADDR_WIDTH+1)) rd_counter (
    .clk(rd_clk),
    .rst_n(rd_rst_n),
    .enable(rd_enable),
    .counter_bin(rd_bin),
    .counter_bin_next(rd_bin_next),
    .counter_gray(rd_gray)
);
```

10.9.2 Cross-Domain Synchronization

```
// Synchronize Gray code pointers across domains
logic [ADDR_WIDTH:0] wr_gray_sync, rd_gray_sync;

// Write domain: synchronize read Gray pointer
synchronizer #(.WIDTH(ADDR_WIDTH+1)) rd_sync (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .data_in(rd_gray),
    .data_out(rd_gray_sync)
);

// Read domain: synchronize write Gray pointer
synchronizer #(.WIDTH(ADDR_WIDTH+1)) wr_sync (
    .clk(rd_clk),
    .rst_n(rd_rst_n),
```

```
    .data_in(wr_gray),
    .data_out(wr_gray_sync)
);
```

10.9.3 FIFO Status Generation

```
// FIFO empty: Gray pointers equal
assign fifo_empty = (rd_gray == wr_gray_sync);

// FIFO full: Binary addresses equal, MSBs different
wire [ADDR_WIDTH-1:0] wr_addr = wr_bin[ADDR_WIDTH-1:0];
wire [ADDR_WIDTH-1:0] rd_addr_sync = gray2bin(rd_gray_sync)
[ADDR_WIDTH-1:0];
wire wr_msb = wr_bin[ADDR_WIDTH];
wire rd_msb_sync = gray2bin(rd_gray_sync)[ADDR_WIDTH];

assign fifo_full = (wr_addr == rd_addr_sync) && (wr_msb !=
rd_msb_sync);
```

10.10 Advanced Features

10.10.1 Almost Full/Empty Flags

```
// Calculate occupancy using binary values
wire [ADDR_WIDTH:0] occupancy = wr_bin - gray2bin(rd_gray_sync);

// Generate status flags
assign almost_full = (occupancy >= ALMOST_FULL_THRESH);
assign almost_empty = (occupancy <= ALMOST_EMPTY_THRESH);
```

10.10.2 Gray-to-Binary Conversion Function

```
function automatic [WIDTH-1:0] gray2bin;
    input [WIDTH-1:0] gray;
    integer i;
    begin
        gray2bin[WIDTH-1] = gray[WIDTH-1];
        for (i = WIDTH-2; i >= 0; i--) begin
            gray2bin[i] = gray2bin[i+1] ^ gray[i];
        end
    end
endfunction
```

10.11 Design Examples

10.11.1 1. 8-bit Asynchronous FIFO Pointer

```
parameter FIFO_DEPTH = 256;
parameter ADDR_WIDTH = $clog2(FIFO_DEPTH);
```

```
parameter PTR_WIDTH = ADDR_WIDTH + 1;  // Extra bit for full detection

counter_bingray #(
    .WIDTH(PTR_WIDTH)
) fifo_wr_ptr (
    .clk(wr_clk),
    .rst_n(async_rst_n),
    .enable(wr_enable && !fifo_full),
    .counter_bin(wr_ptr_bin),
    .counter_bin_next(wr_ptr_bin_next),
    .counter_gray(wr_ptr_gray)
);
```

10.11.2 2. Clock Domain Crossing Counter

```
// Source domain
counter_bingray #(.WIDTH(8)) src_counter (
    .clk(src_clk),
    .rst_n(src_rst_n),
    .enable(src_enable),
    .counter_bin(src_bin),
    .counter_bin_next(),
    .counter_gray(src_gray)
);

// Destination domain receives synchronized Gray value
logic [7:0] dest_gray_sync;
synchronizer #(.WIDTH(8)) sync_inst (
    .clk(dest_clk),
    .rst_n(dest_rst_n),
    .data_in(src_gray),
    .data_out(dest_gray_sync)
);
```

10.12 Verification Strategy

10.12.1 Test Scenarios

1. **Sequential Counting**: Verify both outputs increment correctly
2. **Gray Code Properties**: Ensure single-bit changes between adjacent values
3. **Reset Behavior**: Confirm both outputs reset to zero
4. **Enable Control**: Test hold behavior when disabled
5. **Rollover**: Verify proper wrap-around from maximum value

10.12.2 Coverage Points

```
covergroup counter_bingray_cg @(posedge clk);
    cp_binary: coverpoint counter_bin {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_gray: coverpoint counter_gray {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_enable: coverpoint enable {
        bins enabled = {1};
        bins disabled = {0};
    }

    cp_transitions: coverpoint counter_bin {
        bins transitions[] = ([0:2**WIDTH-2] => [1:2**WIDTH-1]);
        bins rollover = (2**WIDTH-1 => 0);
    }
endgroup
```

10.12.3 Assertions

```
// Verify Gray code has single bit changes
property gray_single_bit_change;
    @(posedge clk) disable iff (!rst_n)
    enable && !$isunknown(counter_gray) |->
    $countones(counter_gray ^ $past(counter_gray)) <= 1;
endproperty

assert property (gray_single_bit_change);

// Verify binary and Gray relationship
property bin_gray_relationship;
    @(posedge clk) disable iff (!rst_n)
    counter_gray == (counter_bin ^ (counter_bin >> 1));
endproperty

assert property (bin_gray_relationship);
```

10.13 Synthesis Considerations

10.13.1 Resource Utilization

- **Flip-Flops**: 2×WIDTH (separate for binary and Gray)
- **LUTs**: Increment logic + XOR tree for Gray conversion

- **Critical Path**: Through binary increment and Gray conversion

10.13.2 Optimization Techniques

```
// Optional: Pipeline Gray conversion for high speed
logic [WIDTH-1:0] counter_gray_pipe;
always_ff @(posedge clk) begin
    counter_gray_pipe <= w_counter_gray;
end
```

10.13.3 Tool-Specific Attributes

```
(* ASYNC_REG = "TRUE" *) logic [WIDTH-1:0] counter_gray; // Xilinx
// synthesis attribute ASYNC_REG of counter_gray is "TRUE"  //
Altera/Intel
```

10.14 Performance Characteristics

10.14.1 Maximum Frequency

- **Typical**: 200-400 MHz in modern FPGAs
- **Limitation**: Binary increment + Gray conversion logic depth
- **Optimization**: Pipeline for higher frequencies

10.14.2 Power Consumption

- **Dynamic**: Proportional to switching activity
- **Static**: Minimal for CMOS technology
- **Optimization**: Clock gating when enable is inactive

10.15 Common Applications

1. **Asynchronous FIFO Pointers**: Primary use case
2. **Clock Domain Crossing Counters**: Safe multi-domain counting
3. **Position Encoders**: Mechanical/optical encoder interfaces
4. **State Machine Counters**: When glitch-free operation required
5. **Memory Address Generation**: For dual-port memory systems

10.16 Troubleshooting Guide

10.16.1 Common Issues

1. **Metastability**: Ensure proper synchronization of Gray values
2. **Timing Violations**: Pipeline Gray conversion if needed
3. **Incorrect FIFO Status**: Verify Gray-to-binary conversion

4. **Reset Skew**: Use proper reset synchronization

10.16.2 Debug Techniques

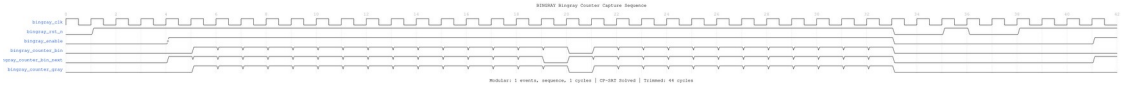
1. **Simulation**: Verify Gray code properties with assertions
2. **Logic Analyzer**: Capture actual transitions in hardware
3. **Timing Analysis**: Check setup/hold requirements
4. **Cross-Domain Checks**: Verify synchronizer timing

10.17 WaveDrom Visualization

High-quality waveforms showcasing Binary-Gray counter operation are available!

The following timing diagrams demonstrate the dual-output counter design across 4 key scenarios:

10.17.1 Scenario 1: Binary vs Gray Code Comparison

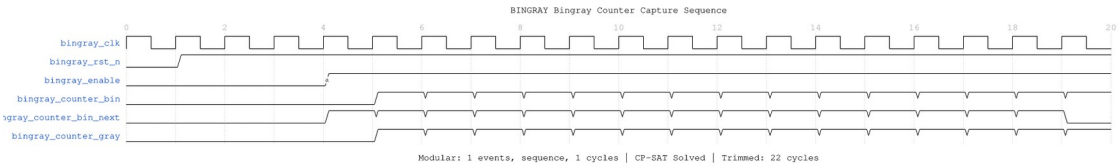


BinGray Comparison

WaveJSON: bingray_counter_binary_vs_gray.json

Side-by-side comparison of both outputs: - Binary: Normal sequential counting (0→1→2→3→...) - Gray: Single-bit transitions between values - Shows full cycle demonstrating encoding differences - Illustrates why Gray code is CDC-safe

10.17.2 Scenario 2: Single-Bit Transitions (CDC Safety) ⭐ KEY FEATURE

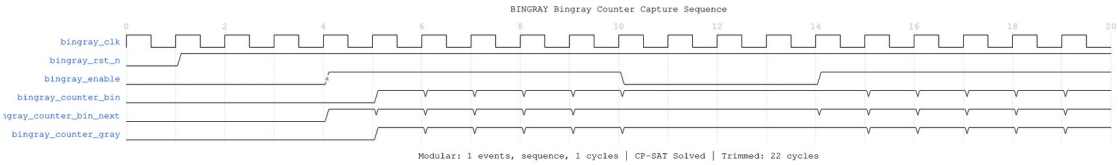


BinGray Single-Bit Transitions

WaveJSON: bingray_counter_single_bit_transitions.json

Gray code CDC safety property: - Each Gray transition changes EXACTLY one bit - Hamming distance = 1 between adjacent values - **Critical for fifo_async CDC mechanism** - Prevents metastability in clock domain crossing

10.17.3 Scenario 3: Lookahead Signal (counter_bin_next)

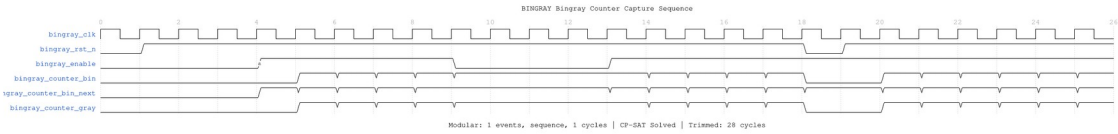


BinGray Lookahead

WaveJSON: bingray_counter_lookahead.json

Combinational lookahead feature: - Predicts next binary value one cycle ahead - Useful for FIFO full/empty prediction - Shows enable gating behavior - Enables early decision-making

10.17.4 Scenario 4: Enable and Reset Control



BinGray Enable and Reset

WaveJSON: bingray_counter_enable_reset.json

Control signal behavior: - Both outputs hold when enable=0 - Asynchronous reset to zero - Clean state transitions - Reset during counting demonstration

---

To regenerate these waveforms:

```
pytest val/common/test_counter_bingray_wavedrom.py -v
# Then convert JSON to PNG:
cd docs/markdown/assets/WAVES/counter_bingray
for f in *.json; do wavedrom-cli -i "$f" -p "${f%.json}.png"; done
```

What Makes Binary-Gray Counters Special:

The waveforms highlight the unique dual-output design: - **Dual Encoding**: Binary for arithmetic, Gray for CDC safety - **Single-Bit Transitions**: Gray code changes one bit at a time - **Lookahead**: counter_bin_next provides one-cycle prediction - **CDC-Safe**: Safe for asynchronous clock domain crossing

Relationship to fifo_async:

Binary-Gray counters are the foundation of the standard `fifo_async` module: - **fifo_async** uses this counter for read/write pointers - Gray outputs cross clock domains safely - Binary outputs

used for arithmetic (occupancy, address generation) - Logarithmic width (`$clog2(DEPTH) + 1`) for resource efficiency

Comparison Table:

Feature	BinGray Counter	Johnson Counter
Output Width	`$clog2(DEPTH) + 1` bits	`DEPTH` bits
CDC Method	Standard Gray code	Johnson code
Depth Support	Power-of-2 only	Any even number
Resource Efficiency	Logarithmic (better for large depths)	Linear
Conversion	XOR tree (simple)	Position detection (complex)
Used In	`fifo_async` (standard)	`fifo_async_div2` (flexible)

Comparison with Other Modules:

- `test_counter_johnson_wavedrom.py` - Johnson counter (even depths, linear width)
- `test_fifo_async_wavedrom.py` - BinGray counter in action (async FIFO, power-of-2)
- `test_fifo_async_div2_wavedrom.py` - Johnson counter in action (async FIFO, even depths)

10.18 Test and Verification

Comprehensive Test Suite: - `val/common/test_counter_bingray.py` - Full functional verification - `val/common/test_counter_bingray_wavedrom.py` - WaveDrom timing diagrams ⭐

Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_counter_bingray.py -v

# WaveDrom waveform generation
pytest val/common/test_counter_bingray_wavedrom.py -v
```

10.19 Navigation

- **← Back to RTLCommon Index**
- **← Back to Main Documentation Index**

11 Clock Pulse Generator - Comprehensive Documentation

11.1 Overview

The `clock_pulse` module generates periodic pulse signals with configurable width (period). It creates a single-cycle pulse every WIDTH clock cycles, making it ideal for timing generation, heartbeat signals, sampling triggers, and periodic event generation in digital systems.

11.2 Module Declaration

```
module clock_pulse #(
    parameter int WIDTH = 10  // Width of the generated pulse in clock
cycles
) (
    input  logic clk,    // Input clock signal
    input  logic rst_n,  // Input reset signal
    output logic pulse   // Output pulse signal
);
```

11.3 Parameters

11.3.1 WIDTH

- **Type**: `int`
- **Default**: `10`
- **Description**: Period of the pulse generation in clock cycles
- **Range**: 2 to 2^32-1 (practical range)
- **Impact**: Determines pulse frequency (f_clk / WIDTH)
- **Pulse Width**: Always exactly 1 clock cycle
- **Duty Cycle**: 1/WIDTH (e.g., 10% for WIDTH=10)

11.4 Ports

11.4.1 Inputs

Port	Width	Type	Description
`clk`	1	`logic`	System clock input
`rst_n`	1	`logic`	Active-low asynchronous

Port	Width	Type	Description
			reset

11.4.2 Outputs

Port	Width	Type	Description
pulse	1	logic	Periodic pulse output

11.5 Architecture and Implementation

11.5.1 Internal Counter

```
logic [WIDTH-1:0] r_counter;
logic [WIDTH-1:0] w_width_minus_one;

// Create a properly sized constant
assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;
```

11.5.2 Core Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse     <= 'b0;
    end else begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1'b1;
        else
            r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one);
    end
end
```

11.5.3 Operation Principles

1. **Counter**: Free-running counter from 0 to WIDTH-1
2. **Pulse Generation**: Pulse asserted when counter reaches WIDTH-1
3. **Auto-Reset**: Counter wraps to 0 after reaching maximum
4. **Synchronous**: All operations synchronized to input clock
5. **Single Cycle**: Pulse duration is exactly one clock cycle

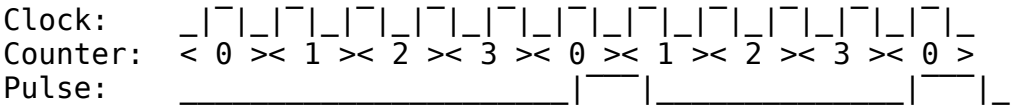
11.5.4 Timing Characteristics

- **Period**: WIDTH clock cycles

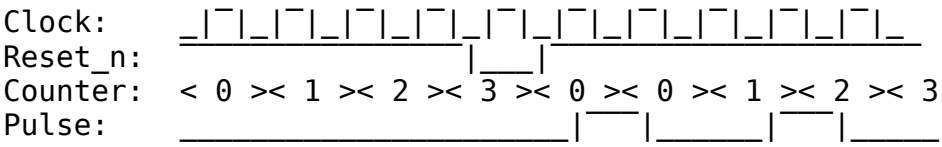
- **Frequency**: f_clk / WIDTH
- **Pulse Width**: 1 clock cycle
- **Duty Cycle**: 1/WIDTH
- **Phase**: Pulse occurs on last count (WIDTH-1)

11.6 Timing Diagrams

11.6.1 Basic Operation (WIDTH=4)



11.6.2 Reset Behavior



11.6.3 Different WIDTH Values

WIDTH	Period	Duty Cycle	Use Case
2	2 cycles	50%	Clock divider
4	4 cycles	25%	Sampling
10	10 cycles	10%	Timing reference
100	100 cycles	1%	Heartbeat
1000	1000 cycles	0.1%	Slow events

11.7 Design Examples and Applications

11.7.1 1. Heartbeat and Status Indicators

```
module system_heartbeat #(
    parameter int CLOCK_FREQ_HZ = 100_000_000,  // 100MHz
    parameter int HEARTBEAT_FREQ_HZ = 1          // 1Hz heartbeat
) (
    input  logic clk,
    input  logic rst_n,
    input  logic system_active,
    output logic heartbeat_led,
    output logic status_ok
);
```

```
    localparam int HEARTBEAT_WIDTH = CLOCK_FREQ_HZ /
HEARTBEAT_FREQ_HZ;

    logic heartbeat_pulse;

    // Generate 1Hz heartbeat pulse
    clock_pulse #(
        .WIDTH(HEARTBEAT_WIDTH)
    ) heartbeat_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(heartbeat_pulse)
    );

    // LED control with heartbeat
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            heartbeat_led <= 1'b0;
        end else if (heartbeat_pulse) begin
            heartbeat_led <= ~heartbeat_led;  // Toggle LED
        end
    end

    // System status based on heartbeat activity
    logic [7:0] heartbeat_monitor;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            heartbeat_monitor <= 8'h00;
        end else begin
            heartbeat_monitor <= {heartbeat_monitor[6:0],
heartbeat_pulse};
        end
    end

    assign status_ok = system_active && |heartbeat_monitor;

endmodule
```

11.7.22. Sampling and Data Acquisition

```
module data_sampler #(
    parameter int SAMPLE_RATE_KHZ = 48,
    parameter int CLOCK_FREQ_MHZ = 100,
    parameter int DATA_WIDTH = 16
) (
    input  logic                    clk,
```

```
    input  logic                  rst_n,
    input  logic                  enable,
    input  logic [DATA_WIDTH-1:0] analog_data,

    output logic [DATA_WIDTH-1:0] sampled_data,
    output logic                  sample_valid,
    output logic                  buffer_full
);

    localparam int SAMPLE_WIDTH = (CLOCK_FREQ_MHZ * 1000) /
SAMPLE_RATE_KHZ;

    logic sample_trigger;

    // Generate sampling trigger
    clock_pulse #(
        .WIDTH(SAMPLE_WIDTH)
    ) sample_clock (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(sample_trigger)
    );

    // Sample data on trigger
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            sampled_data <= '0;
            sample_valid <= 1'b0;
        end else if (enable && sample_trigger) begin
            sampled_data <= analog_data;
            sample_valid <= 1'b1;
        end else begin
            sample_valid <= 1'b0;
        end
    end

    // Buffer management (simplified)
    logic [7:0] buffer_count;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            buffer_count <= 8'h00;
        end else if (sample_valid) begin
            buffer_count <= buffer_count + 1;
        end
    end

    assign buffer_full = &buffer_count;  // Full at 255 samples
```

```
endmodule
```

11.7.33. Communication Protocol Timing

```
module uart_baud_generator #(
    parameter int CLOCK_FREQ_HZ = 100_000_000,
    parameter int BAUD_RATE = 115200
) (
    input  logic clk,
    input  logic rst_n,
    input  logic enable,

    output logic baud_tick,      // 1x baud rate
    output logic baud_tick_16x   // 16x baud rate for oversampling
);

    localparam int BAUD_WIDTH = CLOCK_FREQ_HZ / BAUD_RATE;
    localparam int BAUD_16X_WIDTH = CLOCK_FREQ_HZ / (BAUD_RATE * 16);

    logic baud_pulse, baud_16x_pulse;

    // 1x baud rate generator
    clock_pulse #(
        .WIDTH(BAUD_WIDTH)
    ) baud_1x_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(baud_pulse)
    );

    // 16x baud rate generator
    clock_pulse #(
        .WIDTH(BAUD_16X_WIDTH)
    ) baud_16x_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(baud_16x_pulse)
    );

    // Gate with enable
    assign baud_tick = enable ? baud_pulse : 1'b0;
    assign baud_tick_16x = enable ? baud_16x_pulse : 1'b0;

endmodule
```

11.7.44. Memory Refresh Controller

```
module dram_refresh_controller #(
    parameter int REFRESH_PERIOD_US = 64,     // 64µs refresh period
    parameter int CLOCK_FREQ_MHZ = 100,       // 100MHz system clock
    parameter int REFRESH_ROWS = 8192         // Number of rows to
refresh
) (
    input  logic        clk,
    input  logic        rst_n,
    input  logic        refresh_enable,

    output logic        refresh_request,
    output logic [12:0] refresh_row_addr,
    output logic        refresh_active
);

    localparam int REFRESH_WIDTH = (REFRESH_PERIOD_US *
CLOCK_FREQ_MHZ) / REFRESH_ROWS;

    logic refresh_pulse;
    logic [12:0] row_counter;

    // Generate refresh timing
    clock_pulse #(
        .WIDTH(REFRESH_WIDTH)
    ) refresh_timer (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(refresh_pulse)
    );

    // Row address counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            row_counter <= 13'h0000;
        end else if (refresh_pulse && refresh_enable) begin
            if (row_counter >= REFRESH_ROWS - 1)
                row_counter <= 13'h0000;
            else
                row_counter <= row_counter + 1;
        end
    end

    // Output assignments
    assign refresh_request = refresh_pulse && refresh_enable;
    assign refresh_row_addr = row_counter;
    assign refresh_active = refresh_enable;
```

```
endmodule
```

11.7.55. Multi-Rate Pulse Generation

```
module multi_rate_pulse_generator (
    input  logic       clk,
    input  logic       rst_n,
    input  logic [2:0] rate_select,    // Select pulse rate

    output logic       pulse_fast,     // Fast pulse output
    output logic       pulse_medium,   // Medium pulse output
    output logic       pulse_slow,     // Slow pulse output
    output logic       pulse_config    // Configurable pulse output
);

    logic [31:0] config_width;

    // Rate selection for configurable output
    always_comb begin
        case (rate_select)
            3'b000: config_width = 32'd10;     // Very fast
            3'b001: config_width = 32'd100;    // Fast
            3'b010: config_width = 32'd1000;   // Medium
            3'b011: config_width = 32'd10000;  // Slow
            3'b100: config_width = 32'd100000; // Very slow
            default: config_width = 32'd1000;  // Default medium
        endcase
    end

    // Fixed rate pulse generators
    clock_pulse #(.WIDTH(50)) fast_gen (
        .clk(clk), .rst_n(rst_n), .pulse(pulse_fast)
    );

    clock_pulse #(.WIDTH(500)) medium_gen (
        .clk(clk), .rst_n(rst_n), .pulse(pulse_medium)
    );

    clock_pulse #(.WIDTH(5000)) slow_gen (
        .clk(clk), .rst_n(rst_n), .pulse(pulse_slow)
    );

    // Configurable rate generator (requires parameterizable module)
    configurable_pulse_gen config_gen (
        .clk(clk),
        .rst_n(rst_n),
        .width_config(config_width),
```

```
        .pulse(pulse_config)
    );

endmodule

// Configurable pulse generator module
module configurable_pulse_gen #(
    parameter int MAX_WIDTH = 1000000
) (
    input  logic [31:0] width_config,
    input  logic        clk,
    input  logic        rst_n,
    output logic        pulse
);

    logic [31:0] r_counter;
    logic [31:0] w_width_minus_one;

    assign w_width_minus_one = (width_config > 0) ? (width_config - 1)
: 0;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 32'h00000000;
            pulse <= 1'b0;
        end else begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1;
            else
                r_counter <= 32'h00000000;

            pulse <= (r_counter == w_width_minus_one) && (width_config
> 0);
        end
    end

endmodule
```

11.7.66. Test Pattern Generation

```
module test_pattern_generator (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       test_enable,
    input  logic [3:0] pattern_select,

    output logic [7:0] test_data,
    output logic       test_valid,
```

```
    output logic       pattern_complete
);

    logic update_pulse;
    logic [7:0] pattern_counter;

    // Generate test data update rate
    clock_pulse #(.WIDTH(16)) pattern_clock (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(update_pulse)
    );

    // Pattern counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pattern_counter <= 8'h00;
        end else if (test_enable && update_pulse) begin
            pattern_counter <= pattern_counter + 1;
        end
    end

    // Test pattern generation
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            test_data <= 8'h00;
            test_valid <= 1'b0;
        end else if (test_enable && update_pulse) begin
            case (pattern_select)
                4'h0: test_data <= pattern_counter;
// Incrementing
                4'h1: test_data <= ~pattern_counter;
// Decrementing
                4'h2: test_data <= {pattern_counter[0], 7'b0000000};
// Walking 1
                4'h3: test_data <= ~{pattern_counter[0], 7'b0000000};
// Walking 0
                4'h4: test_data <= 8'hAA;
// Alternating
                4'h5: test_data <= 8'h55;
// Alternating inv
                4'h6: test_data <= 8'hFF;
// All ones
                4'h7: test_data <= 8'h00;
// All zeros
                default: test_data <= pattern_counter;
// Default
            endcase
```

```
            test_valid <= 1'b1;
        end else begin
            test_valid <= 1'b0;
        end
    end

    assign pattern_complete = test_enable && (&pattern_counter) &&
update_pulse;

endmodule
```

11.8 Advanced Features and Variations

11.8.1 1. Enable-Controlled Pulse Generator

```
module pulse_generator_with_enable #(
    parameter int WIDTH = 10
) (
    input  logic clk,
    input  logic rst_n,
    input  logic enable,      // Pulse generation enable
    input  logic pause,       // Pause counting (hold current state)
    output logic pulse,
    output logic [WIDTH-1:0] count_value  // Current counter value
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;
    assign count_value = r_counter;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse <= 1'b0;
        end else if (enable && !pause) begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1'b1;
            else
                r_counter <= 'b0;

            pulse <= (r_counter == w_width_minus_one);
        end else begin
            pulse <= 1'b0;  // No pulse when disabled or paused
        end
    end
```

```
endmodule
```

11.8.22. Variable Width Pulse Generator

```
module variable_pulse_generator #(
    parameter int MAX_WIDTH = 65536
) (
    input  logic                          clk,
    input  logic                          rst_n,
    input  logic [$clog2(MAX_WIDTH):0]  pulse_width,   // Configurable
width
    input  logic                          load_width,    // Load new
width
    output logic                          pulse,
    output logic                          width_loaded
);

    logic [$clog2(MAX_WIDTH):0] r_counter;
    logic [$clog2(MAX_WIDTH):0] r_width;
    logic [$clog2(MAX_WIDTH):0] w_width_minus_one;

    assign w_width_minus_one = (r_width > 0) ? (r_width - 1) : 0;

    // Width register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_width <= MAX_WIDTH[$clog2(MAX_WIDTH):0];  // Default
width
            width_loaded <= 1'b0;
        end else if (load_width) begin
            r_width <= (pulse_width > 0) ? pulse_width : 1;  //
Minimum width = 1
            width_loaded <= 1'b1;
        end else begin
            width_loaded <= 1'b0;
        end
    end

    // Counter and pulse generation
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse <= 1'b0;
        end else begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1;
            else
```

```
                r_counter <= 'b0;

            pulse <= (r_counter == w_width_minus_one) && (r_width >
0);
        end
    end

endmodule
```

11.8.33. Multi-Phase Pulse Generator

```
module multi_phase_pulse_generator #(
    parameter int WIDTH = 12,
    parameter int PHASES = 4
) (
    input  logic             clk,
    input  logic             rst_n,
    input  logic             enable,
    output logic [PHASES-1:0] phase_pulses
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;
    logic [WIDTH-1:0] phase_thresholds [PHASES];

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

    // Calculate phase thresholds
    genvar i;
    generate
        for (i = 0; i < PHASES; i++) begin : calc_phases
            assign phase_thresholds[i] = (WIDTH * i) / PHASES;
        end
    endgenerate

    // Counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
        end else if (enable) begin
            if (r_counter < w_width_minus_one)
                r_counter <= r_counter + 1;
            else
                r_counter <= 'b0;
        end
    end

    // Phase pulse generation
```

```
    generate
        for (i = 0; i < PHASES; i++) begin : gen_phases
            always_ff @(posedge clk or negedge rst_n) begin
                if (!rst_n) begin
                    phase_pulses[i] <= 1'b0;
                end else begin
                    phase_pulses[i] <= enable && (r_counter ==
phase_thresholds[i]);
                end
            end
        end
    endgenerate

endmodule
```

11.8.44. Burst Pulse Generator

```
module burst_pulse_generator #(
    parameter int BURST_LENGTH = 8,
    parameter int BURST_PERIOD = 100,
    parameter int INTER_BURST_DELAY = 1000
) (
    input  logic clk,
    input  logic rst_n,
    input  logic enable,
    input  logic trigger,           // Start burst sequence

    output logic burst_pulse,       // Individual pulses in burst
    output logic burst_active,      // Burst sequence active
    output logic burst_complete     // Burst sequence completed
);

    typedef enum logic [2:0] {
        IDLE,
        BURST_ACTIVE,
        INTER_PULSE_DELAY,
        INTER_BURST_DELAY_ST
    } burst_state_t;

    burst_state_t state;

    logic [$clog2(BURST_LENGTH):0] pulse_count;
    logic [$clog2(BURST_PERIOD):0] period_counter;
    logic [$clog2(INTER_BURST_DELAY):0] delay_counter;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= IDLE;
```

```
            pulse_count <= 0;
            period_counter <= 0;
            delay_counter <= 0;
            burst_pulse <= 1'b0;
            burst_complete <= 1'b0;
        end else begin
            burst_complete <= 1'b0;  // Default

            case (state)
                IDLE: begin
                    burst_pulse <= 1'b0;
                    if (enable && trigger) begin
                        state <= BURST_ACTIVE;
                        pulse_count <= 0;
                        period_counter <= 0;
                        burst_pulse <= 1'b1;  // First pulse
                    end
                end

                BURST_ACTIVE: begin
                    burst_pulse <= 1'b0;
                    pulse_count <= pulse_count + 1;

                    if (pulse_count >= BURST_LENGTH - 1) begin
                        state <= INTER_BURST_DELAY_ST;
                        delay_counter <= 0;
                        burst_complete <= 1'b1;
                    end else begin
                        state <= INTER_PULSE_DELAY;
                        period_counter <= 0;
                    end
                end

                INTER_PULSE_DELAY: begin
                    period_counter <= period_counter + 1;
                    if (period_counter >= BURST_PERIOD - 1) begin
                        state <= BURST_ACTIVE;
                        burst_pulse <= 1'b1;
                    end
                end

                INTER_BURST_DELAY_ST: begin
                    delay_counter <= delay_counter + 1;
                    if (delay_counter >= INTER_BURST_DELAY - 1) begin
                        state <= IDLE;
                    end
                end
            endcase
```

```
        end
    end

    assign burst_active = (state != IDLE);

endmodule
```

11.9 Verification and Testing

11.9.1 Comprehensive Test Bench

```
module tb_clock_pulse;

    parameter WIDTH = 10;

    logic clk, rst_n, pulse;

    // Clock generation
    initial clk = 0;
    always #5ns clk = ~clk; // 100MHz

    // DUT instantiation
    clock_pulse #(.WIDTH(WIDTH)) dut (.*);

    // Test sequence
    initial begin
        rst_n = 0;
        #100ns rst_n = 1;

        // Test basic pulse generation
        test_basic_pulse_generation();

        // Test reset behavior
        test_reset_behavior();

        // Test timing accuracy
        test_timing_accuracy();

        $display("All tests completed!");
        $finish;
    end

    // Test basic pulse generation
    task test_basic_pulse_generation();
        int pulse_count = 0;
        int cycle_count = 0;
        begin
            $display("Testing basic pulse generation (WIDTH=%0d)...",
```

```
WIDTH);

            // Monitor for several pulse periods
            repeat (WIDTH * 3) begin
                @(posedge clk);
                cycle_count++;
                if (pulse) pulse_count++;
            end

            // Verify pulse count
            if (pulse_count == 3) begin
                $display("PASS: Correct number of pulses generated");
            end else begin
                $error("FAIL: Expected 3 pulses, got %0d",
pulse_count);
            end
        end
    endtask

    // Test timing accuracy
    task test_timing_accuracy();
        time pulse_times[10];
        time pulse_periods[9];
        int i;
        real avg_period, expected_period;
        begin
            $display("Testing timing accuracy...");

            expected_period = WIDTH * 10.0; // 10ns clock period

            // Capture pulse timing
            for (i = 0; i < 10; i++) begin
                @(posedge pulse);
                pulse_times[i] = $time;
            end

            // Calculate periods
            for (i = 0; i < 9; i++) begin
                pulse_periods[i] = pulse_times[i+1] - pulse_times[i];
            end

            // Calculate average period
            avg_period = 0;
            for (i = 0; i < 9; i++) begin
                avg_period += pulse_periods[i];
            end
            avg_period = avg_period / 9.0;
```

```
            // Check accuracy (within 1% tolerance)
            if (abs(avg_period - expected_period) < expected_period *
0.01) begin
                $display("PASS: Average period = %.1f ns (expected
%.1f ns)",
                         avg_period, expected_period);
            end else begin
                $error("FAIL: Average period = %.1f ns (expected %.1f
ns)",
                       avg_period, expected_period);
            end
        end
    endtask

    function real abs(real value);
        abs = (value >= 0) ? value : -value;
    endfunction

endmodule
```

11.9.2 Coverage Model

```
covergroup clock_pulse_cg @(posedge clk);

    cp_counter_values: coverpoint dut.r_counter {
        bins zero = {0};
        bins low[] = {[1:WIDTH/4]};
        bins mid[] = {[WIDTH/4+1:3*WIDTH/4]};
        bins high[] = {[3*WIDTH/4+1:WIDTH-2]};
        bins max = {WIDTH-1};
    }

    cp_pulse: coverpoint pulse {
        bins asserted = {1};
        bins deasserted = {0};
    }

    cp_reset: coverpoint rst_n {
        bins reset = {0};
        bins normal = {1};
    }

    // Transition coverage
    cp_pulse_edges: coverpoint pulse {
        bins rising = (0 => 1);
        bins falling = (1 => 0);
        bins stable_low = (0 => 0);
        bins stable_high = (1 => 1);
    }
```

```
    // Cross coverage
    cross_counter_pulse: cross cp_counter_values, cp_pulse;

endcovergroup
```

11.9.3 Formal Properties

```
module clock_pulse_properties;

    // Bind to DUT
    bind clock_pulse clock_pulse_properties props_inst (.*);

    // Property: Pulse occurs at correct count
    property pulse_at_max_count;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |-> pulse;
    endproperty

    // Property: Pulse only occurs at max count
    property pulse_only_at_max;
        @(posedge clk) disable iff (!rst_n)
        pulse |-> (dut.r_counter == WIDTH-1);
    endproperty

    // Property: Counter wraps correctly
    property counter_wrap;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |=> (dut.r_counter == 0);
    endproperty

    // Property: Counter increments
    property counter_increment;
        @(posedge clk) disable iff (!rst_n)
        (dut.r_counter < WIDTH-1) |=>
        (dut.r_counter == $past(dut.r_counter) + 1);
    endproperty

    // Property: Reset behavior
    property reset_behavior;
        @(posedge clk)
        !rst_n |=> (dut.r_counter == 0) && !pulse;
    endproperty

    // Assertions
    assert property (pulse_at_max_count);
    assert property (pulse_only_at_max);
    assert property (counter_wrap);
```

```
    assert property (counter_increment);
    assert property (reset_behavior);

endmodule
```

11.10 Synthesis Considerations

11.10.1 Resource Utilization

WIDTH	Counter Bits	LUTs	FFs	Max Freq
8	3	8	4	500MHz
16	4	12	5	450MHz
32	5	18	6	400MHz
1024	10	28	11	350MHz

11.10.2 Timing Optimization

```
// For high-frequency applications, pipeline the comparison
module clock_pulse_pipelined #(
    parameter int WIDTH = 1000
) (
    input  logic clk,
    input  logic rst_n,
    output logic pulse
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;
    logic compare_result;

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

    // Pipeline the comparison
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            compare_result <= 1'b0;
        end else begin
            compare_result <= (r_counter == w_width_minus_one);
        end
    end

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 'b0;
            pulse <= 1'b0;
        end else begin
```

```
            if (compare_result)
                r_counter <= 'b0;
            else
                r_counter <= r_counter + 1;

            pulse <= compare_result;
        end
    end

endmodule
```

11.11 Common Applications Summary

1. **Timing References**: System heartbeats, watchdog timers
2. **Sampling Systems**: ADC triggers, data acquisition timing
3. **Communication**: Baud rate generation, protocol timing
4. **Memory Systems**: Refresh timing, access scheduling
5. **Test Equipment**: Pattern generation, stimulus timing
6. **Power Management**: Activity monitoring, timeout generation
7. **Display Systems**: Refresh rates, sync generation

11.12 Design Guidelines

1. **Choose Appropriate WIDTH**: Balance between resolution and resource usage
2. **Consider Reset Timing**: Ensure clean startup behavior
3. **Validate Timing**: Verify pulse frequency matches requirements
4. **Plan for Variation**: Consider process, voltage, temperature effects
5. **Monitor Resource Usage**: Large counters can impact timing and area
6. **Use Enables**: Add enable signals for conditional operation
7. **Document Timing**: Clearly specify pulse rates and relationships

The clock pulse generator provides a simple, reliable solution for periodic timing generation, essential for many digital system functions requiring precise, regular timing events.

11.13 Navigation

- ← Back to RTLCommon Index
- ← Back to Main Documentation Index

12 APB Slave CDC

Module: `apb_slave_cdc.sv` **Location:** `rtl/amba/apb/` **Status:** √ Production Ready

12.1 Overview

The APB Slave CDC (Clock Domain Crossing) module provides a complete APB slave interface with integrated clock domain crossing between the APB (pclk) domain and an AXI/GAXI (aclk) domain. This enables safe integration of APB peripherals running at different clock frequencies.

12.1.1 Key Features

- √ **Safe CDC:** Proper clock domain crossing with handshake protocol
- √ **Dual Clock Domains:** APB (pclk) and AXI (aclk) operate independently
- √ **Full APB4/APB5 Support:** Complete protocol compliance
- √ **Command/Response Interface:** Clean GAXI-style backend interface
- √ **Buffered Operation:** Integrated skid buffers for elastic storage

12.2 Module Interface

```
module apb_slave_cdc #(
    parameter int ADDR_WIDTH  = 32,
    parameter int DATA_WIDTH  = 32,
    parameter int STRB_WIDTH  = DATA_WIDTH / 8,
    parameter int PROT_WIDTH  = 3,
    parameter int DEPTH       = 2
) (
    // Clock and Reset
    input  logic              aclk,
    input  logic              aresetn,
    input  logic              pclk,
    input  logic              presetn,

    // APB Slave Interface (pclk domain)
    input  logic              s_apb_PSEL,
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
```

```
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

    // Command Interface (aclk domain)
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface (aclk domain)
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

12.3 Clock Domains

12.3.1 APB Domain (pclk)

- APB slave interface signals
- Typical frequency: 50-200 MHz
- Used by APB master/interconnect

12.3.2 AXI Domain (aclk)

- Command and response interfaces
- Can be faster or slower than pclk
- Used by backend processing logic

Note: The module handles the clock domain crossing safely using handshake-based CDC techniques.

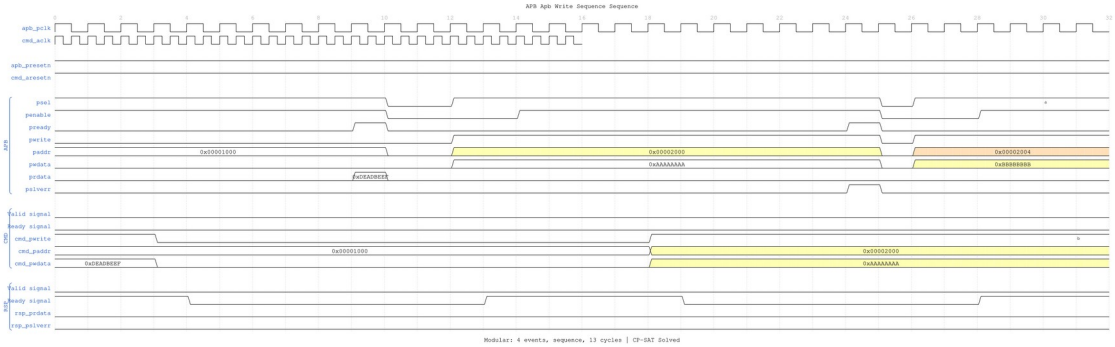
12.4 Waveforms

The following timing diagrams show CDC behavior with **both clock domains visible**:

Clock Configuration: - `apb_pclk`: 100MHz (10ns period) - `cmd_aclk`: 500MHz (2ns period), displayed with `period=0.4` for visual compactness

12.4.1 Scenario 1: Write Transaction with CDC

Shows APB write crossing from pclk domain to aclk domain:



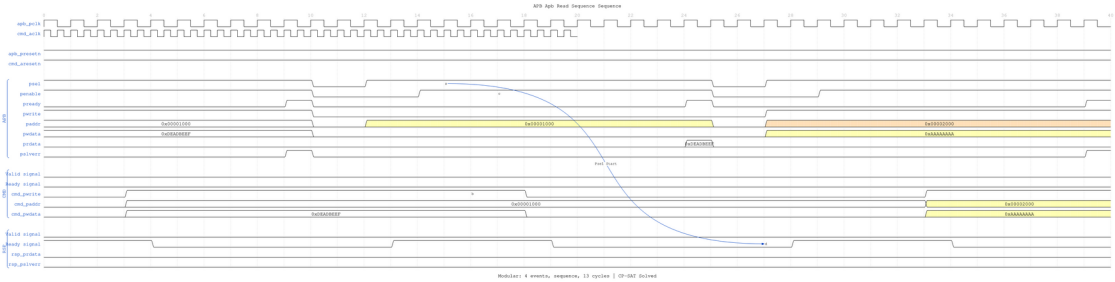
Write CDC

WaveJSON: apb_write_sequence_001.json

Key Observations: - APB transaction in pclk domain - CMD transaction crosses to aclk domain - Note the CDC latency between domains

12.4.2 Scenario 2: Read Transaction with CDC

Shows APB read with response crossing back from aclk to pclk domain:

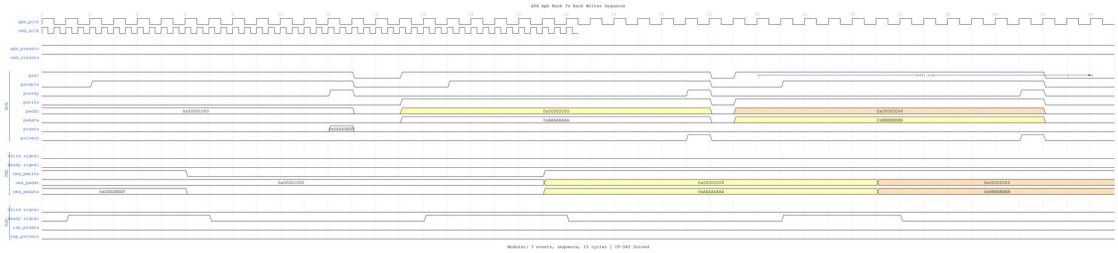


Read CDC

WaveJSON: apb_read_sequence_001.json

Key Observations: - APB read request in pclk domain - CMD crosses to aclk domain - RSP crosses back to pclk domain - Complete round-trip CDC visible

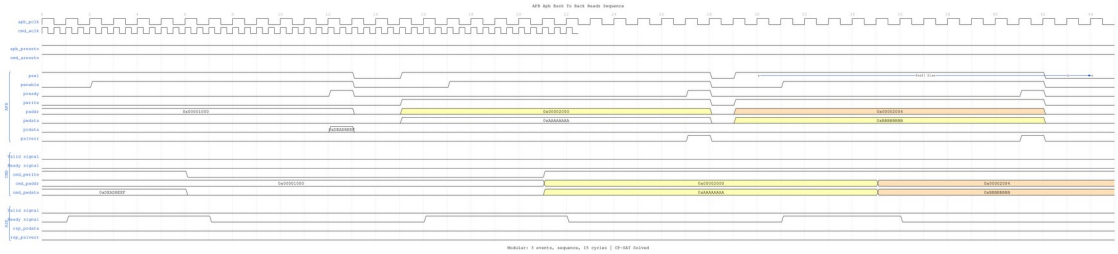
12.4.3 Scenario 3: Back-to-Back Writes with CDC



B2B Writes CDC

WaveJSON: apb_back_to_back_writes_001.json

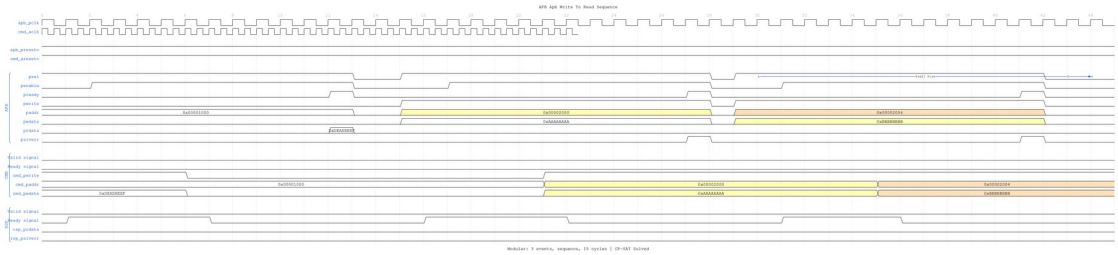
12.4.4 Scenario 4: Back-to-Back Reads with CDC



B2B Reads CDC

WaveJSON: apb_back_to_back_reads_001.json

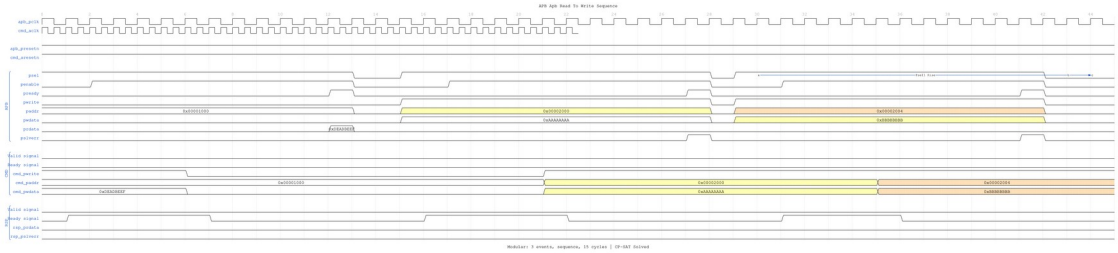
12.4.5 Scenario 5: Write-to-Read Transition with CDC



Write-to-Read CDC

WaveJSON: apb_write_to_read_001.json

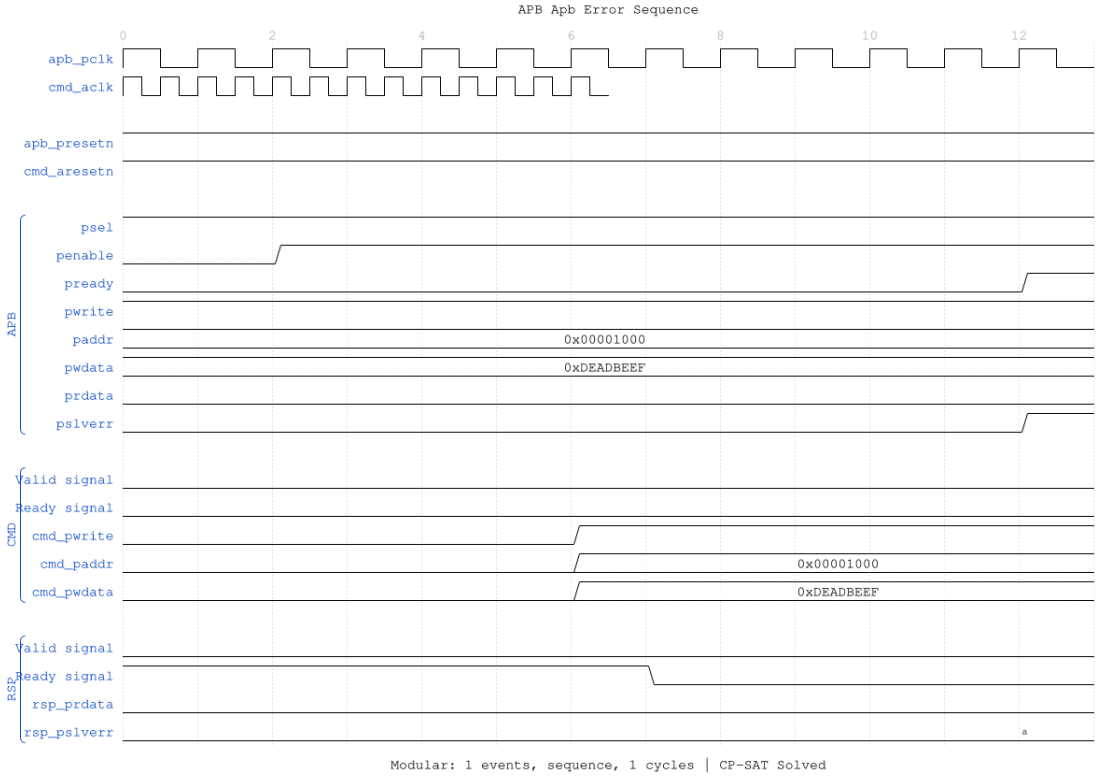
12.4.6 Scenario 6: Read-to-Write Transition with CDC



Read-to-Write CDC

WaveJSON: apb_read_to_write_001.json

12.4.7 Scenario 7: Error Response with CDC



Error CDC

WaveJSON: apb_error_001.json

12.5 Usage Example

```
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_cdc (
    // APB clock domain
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // AXI clock domain
    .aclk         (axi_clk),
    .aresetn      (axi_resetn),

    // APB slave interface (pclk domain)
    .s_apb_PSEL     (apb_psel),
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface (aclk domain)
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),

    // Response interface (aclk domain)
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

12.6 References

- **APB Slave:** apb_slave.md
- **Source:** `rtl/amba/apb/apb_slave_cdc.sv`
- **Tests:** `val/amba/test_apb_slave_cdc.py`
- **WaveDrom Test:** `val/amba/test_apb_slave_cdc.py::test_apb_slave_cdc_wavedrom`

Last Updated: 2025-10-09

12.7 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

13 APB Slave with CDC (Clock-Gated)

Module: `apb_slave_cdc_cg.sv` **Base Module:** apb_slave_cdc **Location:** `rtl/amba/apb/`
Status: √ Production Ready

13.1 Quick Reference

This is the **clock-gated variant** of apb_slave_cdc.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ Clock-Gated Variants Guide

13.2 Summary

The `apb_slave_cdc_cg` module adds power optimization to `apb_slave_cdc` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization

- ✓ **Configurable:** Idle threshold, gating domains, enable/disable
- ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

13.3 Common Parameters

In addition to all apb_slave_cdc parameters:

Parameter	Default	Description
`ENABLE_CLOCK_GATING`	1	Master enable (0=disable, identical to base)
`CG_IDLE_CYCLES`	8	Cycles before clock gating activates
`CG_GATE_*`	1	Domain-specific gating enables

13.4 Quick Usage

```
apb_slave_cdc_cg #(
    // Base module parameters (see apb_slave_cdc.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as apb_slave_cdc
);
```

13.5 Documentation

- **Base Module Functionality:** apb_slave_cdc.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**

- axi4_master_rd_mon_cg.md (AXI4 monitor)
- axil4_master_rd_mon_cg.md (AXIL4 monitor)
- apb_slave_cg.md (APB interface)

13.6 Navigation

- **← Back to APB Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

14 APB5 Slave (Clock Domain Crossing)

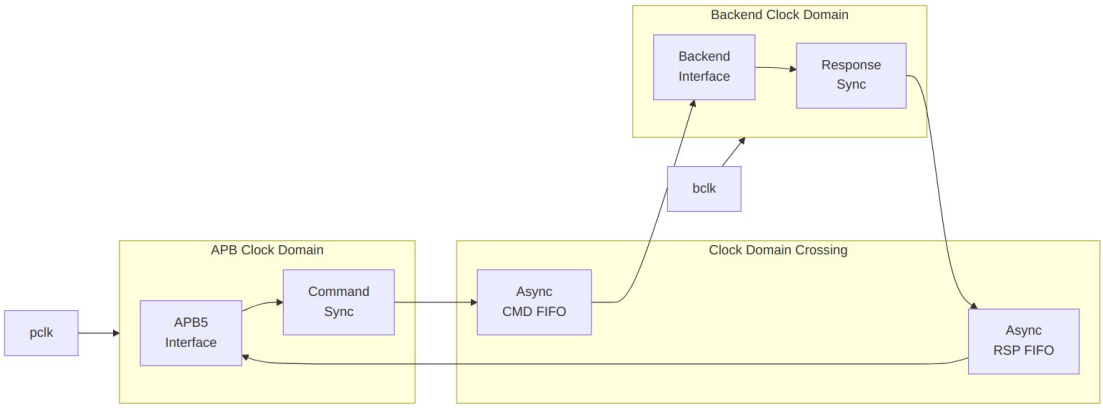
Module: `apb5_slave_cdc.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

14.1 Overview

The APB5 Slave CDC module provides clock domain crossing capability between an APB5 bus clock domain and a backend clock domain. It safely transfers APB5 transactions across asynchronous clock boundaries.

14.1.1 Key Features

- Full APB5 protocol support with CDC
- Asynchronous FIFO-based clock domain crossing
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal crossing
- Configurable FIFO depths for each direction
- Metastability protection



Module Architecture

14.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
CMD_DEPTH	int	4	Command FIFO depth (power of 2)
RSP_DEPTH	int	4	Response FIFO depth (power of 2)

Parameter	Type	Default	Description
SYNC_STAGES	int	2	Synchronizer stages for CDC

14.3 Ports

14.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
bclk	1	Input	Backend clock
bresetn	1	Input	Backend reset (active low)

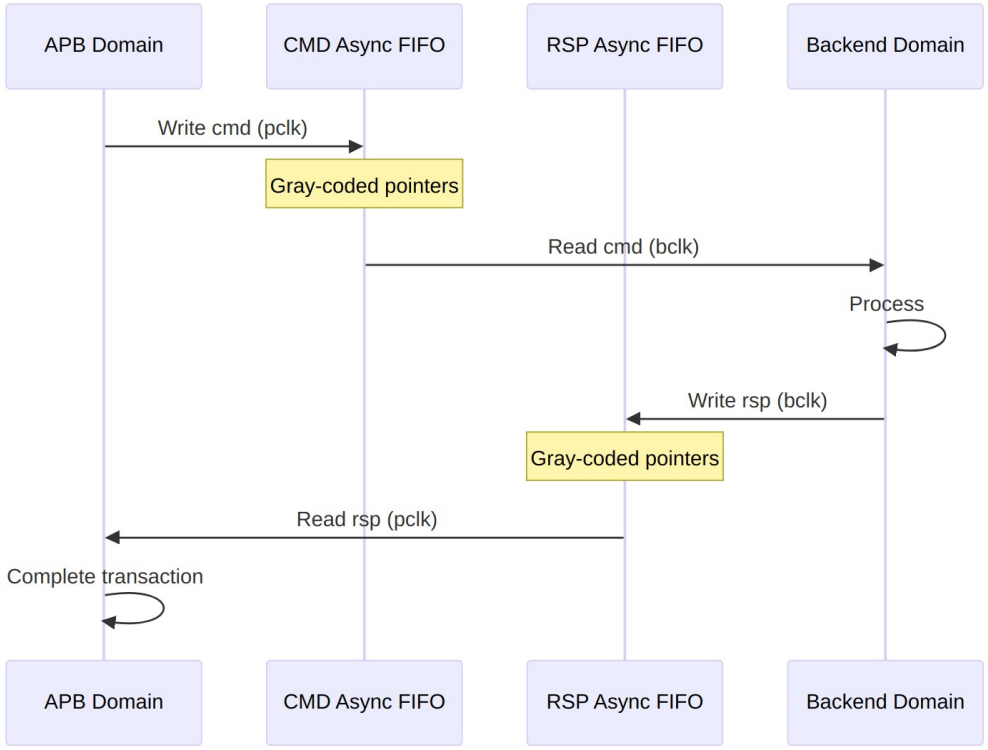
14.3.2 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain.

14.3.3 Backend Interface

Same command/response interface as apb5_slave - operates in `bclk` domain.

14.4 Clock Domain Crossing



CDC Mechanism

14.4.1 Timing Considerations

```
TODO: Wavedrom timing diagram showing:
- pclk
- bclk (different frequency)
- APB transaction signals
- FIFO write/read pointers
- Backend transaction signals
- CDC latency
```

14.5 Usage Example

```
apb5_slave_cdc #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
```

```
    .CMD_DEPTH      (4),
    .RSP_DEPTH      (4),
    .SYNC_STAGES    (2)
) u_apb5_slave_cdc (
    // APB clock domain
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // Backend clock domain
    .bclk           (backend_clk),
    .bresetn        (backend_rst_n),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    // ... other APB signals

    // Backend interface (bclk domain)
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready),
    // ... other backend signals
);
```

14.6 Design Notes

14.6.1 FIFO Depth Sizing

- CMD_DEPTH should handle worst-case APB burst before backend responds
- RSP_DEPTH should handle responses during slow APB clock periods
- Minimum depth: 2 (for proper CDC operation)

14.6.2 Reset Synchronization

- Both resets should be properly synchronized to their respective domains
- Module handles internal reset synchronization for CDC logic

14.6.3 Latency

- Minimum latency: 2-4 cycles per direction (sync stages)
- Additional latency if FIFOs are empty/full

14.7 Related Documentation

- **APB5 Slave** - Base slave without CDC
- **APB5 Slave CDC CG** - CDC with clock gating
- **CDC Handshake** - CDC design patterns

14.8 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

15 APB5 Slave (CDC + Clock-Gated)

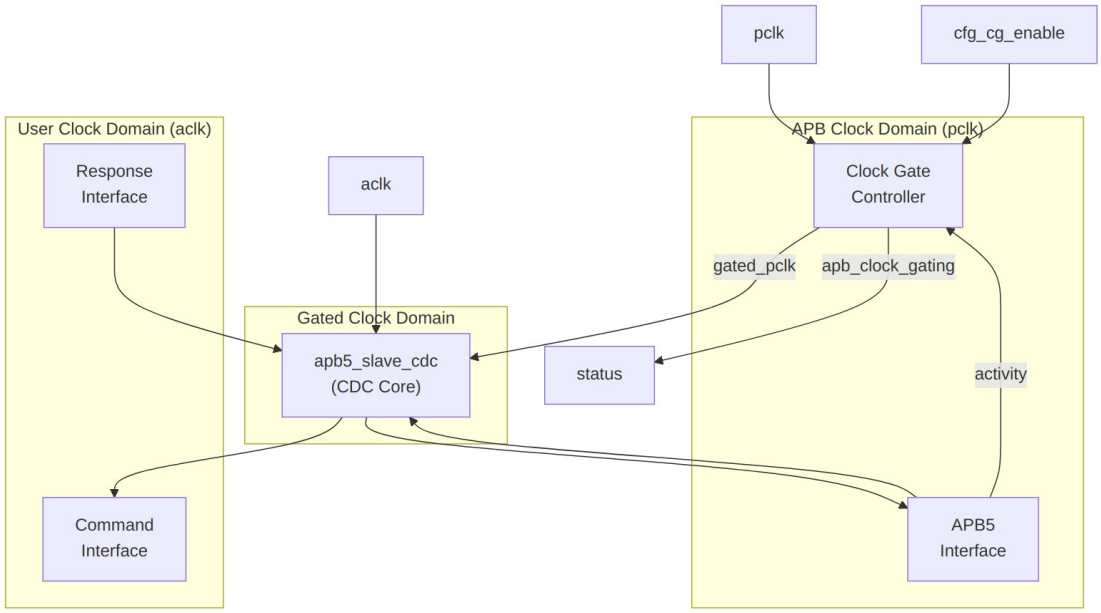
Module: `apb5_slave_cdc_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

15.1 Overview

The APB5 Slave CDC + Clock-Gated module combines clock domain crossing with clock gating for maximum power efficiency. It wraps `apb5_slave_cdc` with clock gating control to reduce power consumption during idle periods while maintaining safe operation across asynchronous clock boundaries.

15.1.1 Key Features

- Full APB5 protocol support with all extensions
- Asynchronous clock domain crossing (APB to backend)
- Clock gating for power reduction during idle
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal handling across domains
- Optional parity support for data integrity
- Automatic wake-up on transaction activity



Module Architecture

15.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Internal buffer

Parameter	Type	Default	Description
			depth
ENABLE_PARITY	bit	0	Enable parity signals
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter

15.3 Ports

15.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	User/backend clock
aresetn	1	Input	User reset (active low)

15.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

15.3.3 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain.

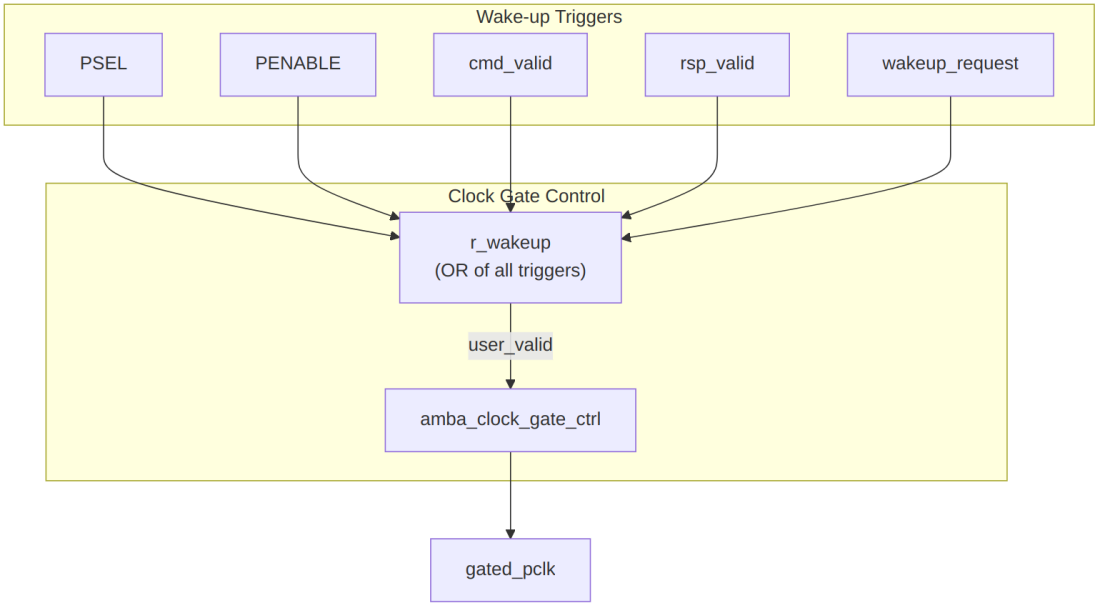
15.3.4 Backend Interface

Same command/response interface as apb5_slave_cdc - operates in `aclk` domain.

15.3.5 Status Outputs

Port	Width	Direction	Description
apb_clock_gating	1	Output	Indicates clock is currently gated
parity_error_wdata	1	Output	Write data parity error detected
parity_error_ctrl	1	Output	Control signal parity error

15.4 Clock Gating and CDC Interaction



Wake-up Logic

15.4.1 Timing Considerations

```
TODO: Wavedrom timing diagram showing:
- pclk (ungated)
- gated_pclk
- aclk (different frequency)
- s_apb_PSEL (wake trigger)
- apb_clock_gating indicator
```

- Transaction flow across CDC with gating
- Wake-up latency from PSEL to clock active

15.5 Usage Example

```
apb5_slave_cdc_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .RUSER_WIDTH        (4),
    .BUSER_WIDTH        (4),
    .DEPTH              (2),
    .ENABLE_PARITY      (0),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cdc_cg (
    // APB clock domain
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // User clock domain
    .aclk               (user_clk),
    .aresetn            (user_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),
    .apb_clock_gating   (slave_clk_gated),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL         (s_apb_psel),
    .s_apb_PENABLE      (s_apb_penable),
    .s_apb_PREADY       (s_apb_pready),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid          (backend_cmd_valid),
    .cmd_ready          (backend_cmd_ready),
    // ... other command signals

    .rsp_valid          (backend_rsp_valid),
    .rsp_ready          (backend_rsp_ready),
    // ... other response signals

    // Wake-up control (aclk domain)
```

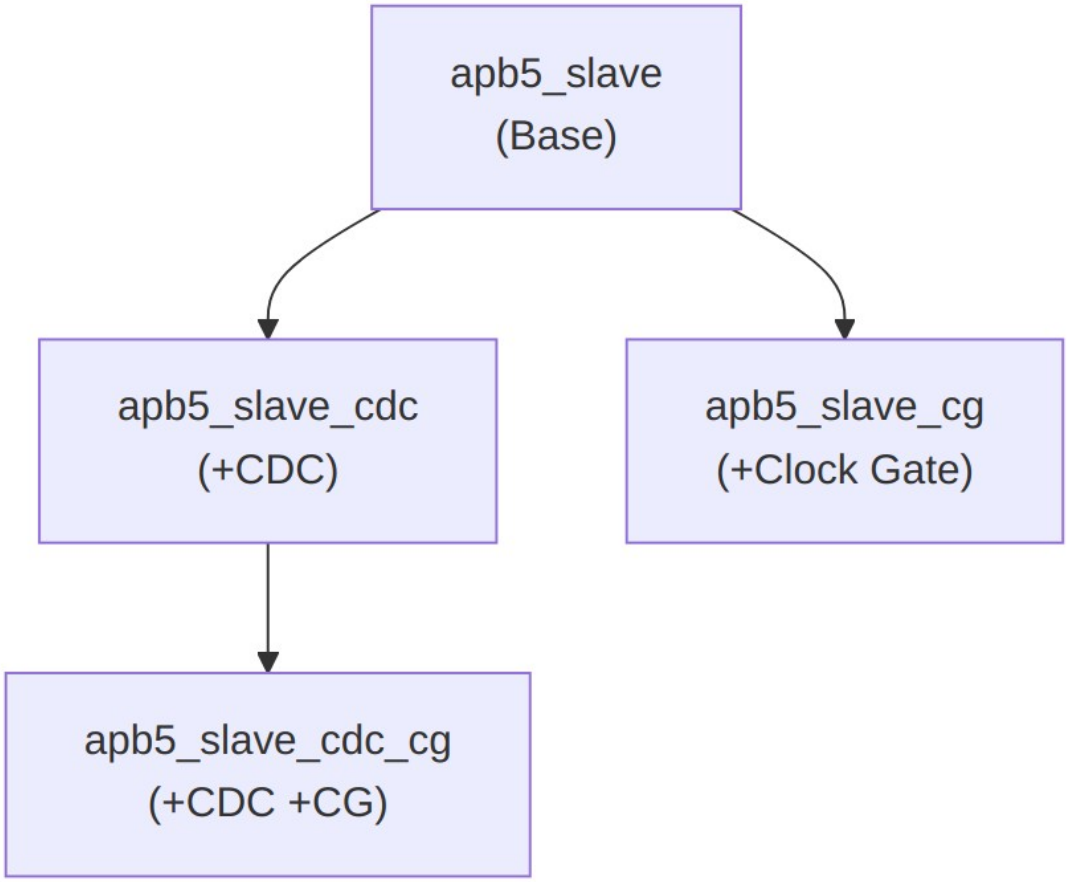
```
    .wakeup_request     (backend_wakeup)
);
```

15.6 Design Notes

15.6.1 Power and Latency Trade-offs

Configuration	Power Savings	Wake-up Latency
CG disabled	None	0 cycles
CG idle=4	Moderate	0 cycles (instant wake)
CG idle=16	Good	0 cycles (instant wake)

Note: Wake-up is instant from PSEL assertion due to combinational wake-up logic.



Combined Feature Hierarchy

15.6.2 Reset Considerations

- APB domain reset (`presetn`) controls APB interface and clock gate
- User domain reset (`aresetn`) controls backend interface
- Both resets must be properly synchronized to their domains
- Module handles internal CDC for reset coordination

15.7 Related Documentation

- **APB5 Slave** - Base slave module
- **APB5 Slave CDC** - CDC variant without clock gating
- **APB5 Slave CG** - Clock gating without CDC

15.8 Navigation

- <- Back to APB5 Index
- <- Back to RTLAmba Index
- <- Back to Main Documentation Index